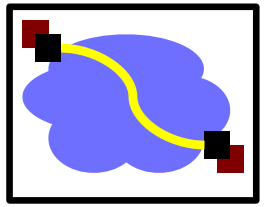


15-440 Distributed Systems

Review

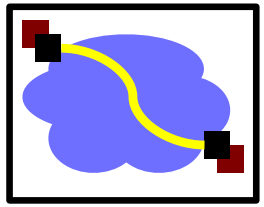
What Is A Distributed System?



"A collection of independent computers that appears to its users as a single coherent system."

- Features:
  - No shared memory – message-based communication
  - Each runs its own local OS
  - Heterogeneity
  - Expandability
- Ideal: to present a single-system image:
  - The distributed system "looks like" a single computer rather than a collection of separate computers.

Definition of a Distributed System



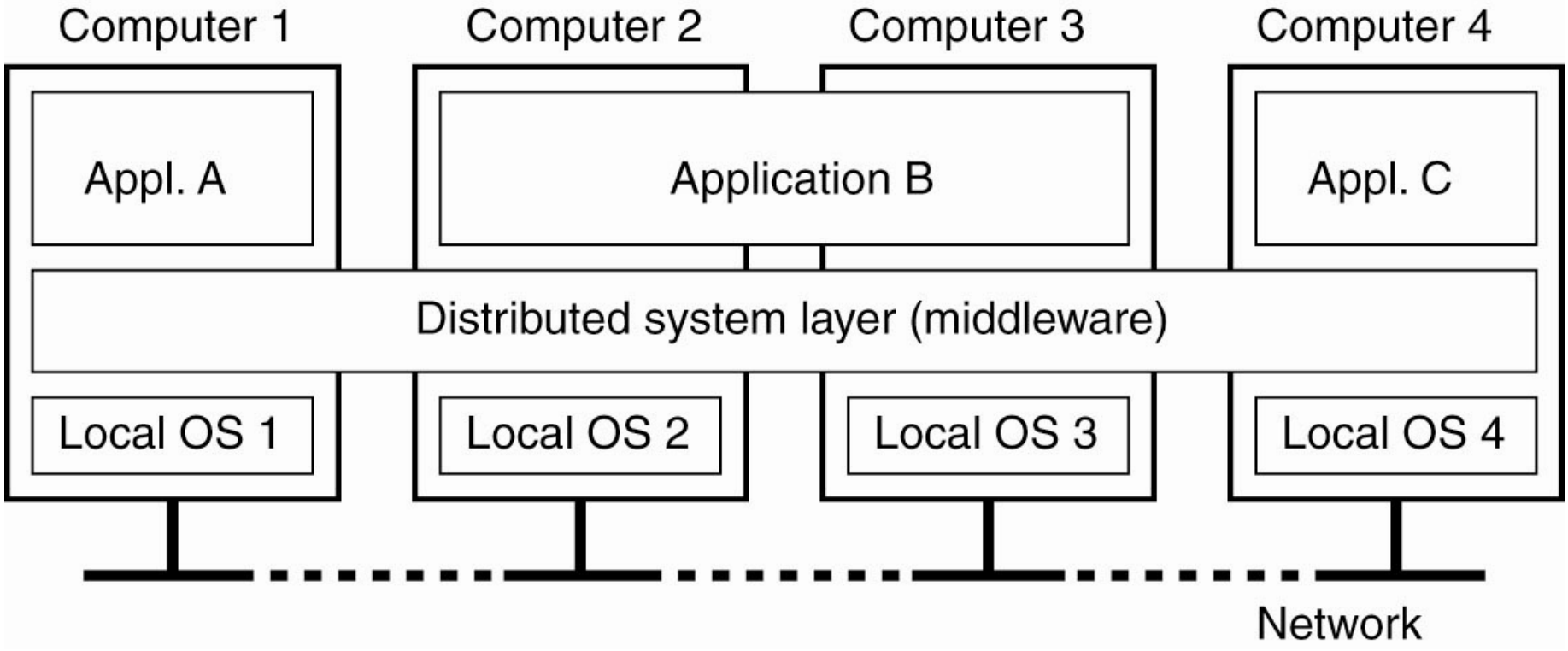
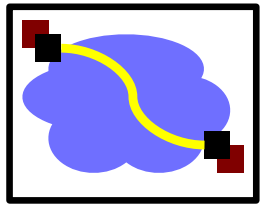


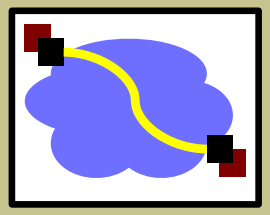
Figure 1-1. A distributed system organized as middleware. The middleware layer runs on all machines, and offers a uniform interface to the system

Distributed Systems: Goals



- Resource Availability: remote access to resources
- Distribution Transparency: single system image
  - Access, Location, Migration, Replication, Failure,…
- Openness: services according to standards (RPC)
- Scalability: size, geographic, admin domains, …

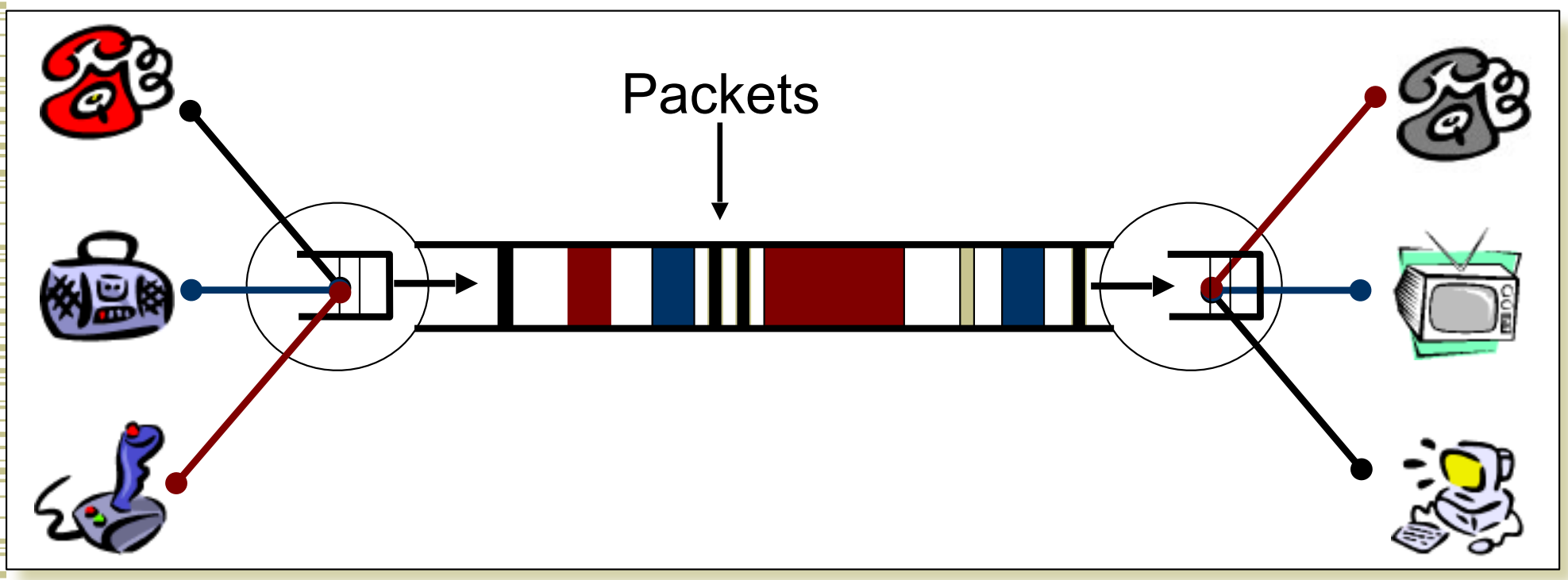
- Example of a Distributed System?
  - Web search on google
  - DNS: decentralized, scalable, robust to failures, ...
  - ...



15-440 Distributed Systems

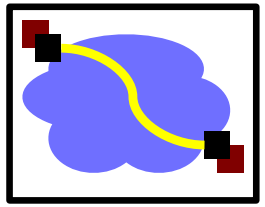
Lecture 2 & 3 – 15-441 in 2 Days

Packet Switching – Statistical Multiplexing



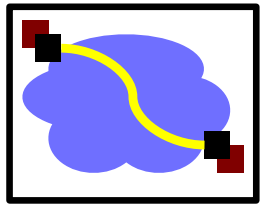
- Switches arbitrate between inputs
- Can send from *any* input that's ready
  - Links never idle when traffic to send
  - (Efficiency!)

Model of a communication channel

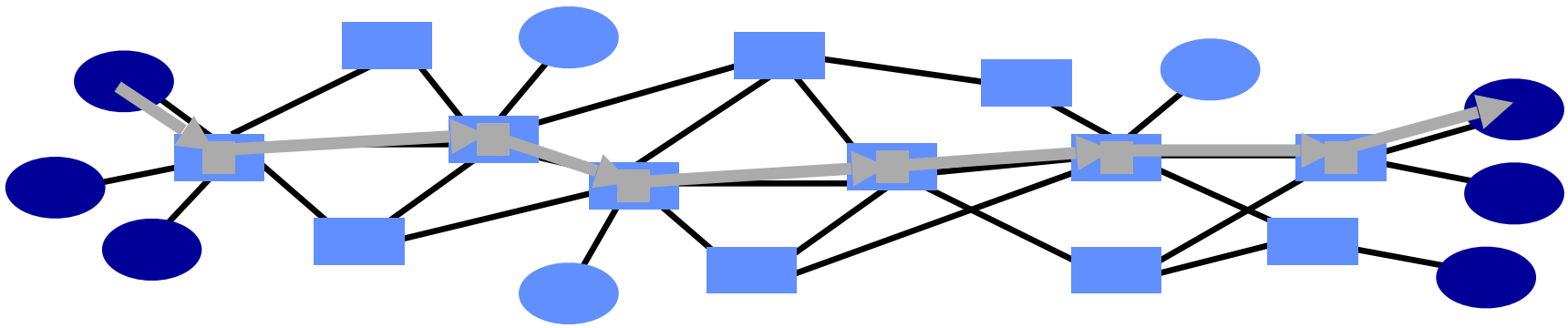


- Latency - how long does it take for the first bit to reach destination
- Capacity - how many bits/sec can we push through? (often termed “bandwidth”)
- Jitter - how much variation in latency?
- Loss / Reliability - can the channel drop packets?
- Reordering

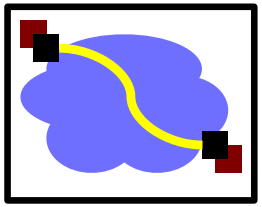
Packet Switching



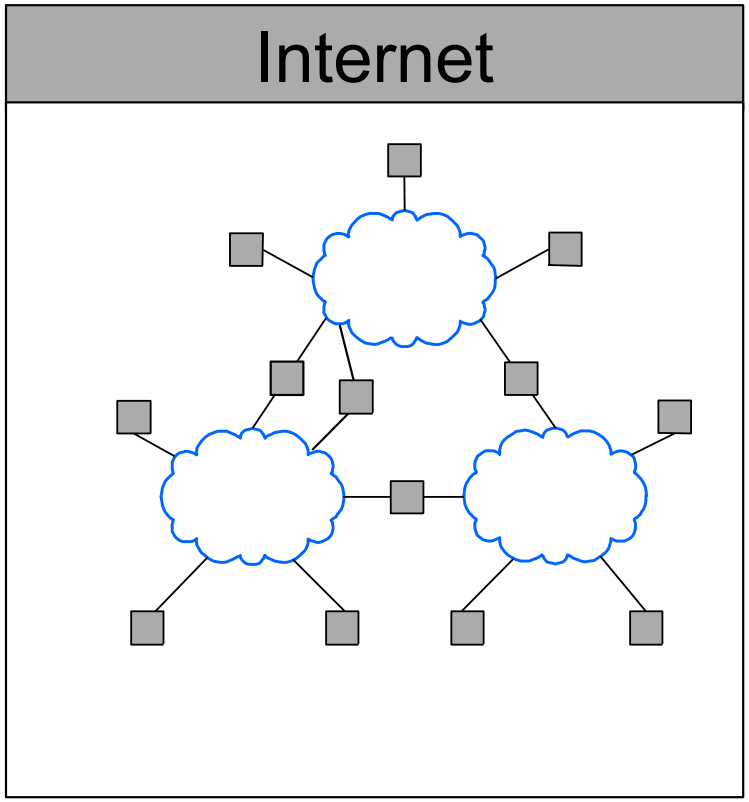
- Source sends information as self-contained packets that have an address.
  - Source may have to break up single message in multiple
- Each packet travels independently to the destination host.
  - Switches use the address in the packet to determine how to forward the packets
  - Store and forward
- Analogy: a letter in surface mail.



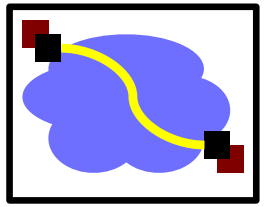
Internet



- An inter-net: a network of networks.
  - Networks are connected using routers that support communication in a hierarchical fashion
  - Often need other special devices at the boundaries for security, accounting, ..
- The Internet: the interconnected set of networks of the Internet Service Providers (ISPs)
  - About 17,000 different networks make up the Internet

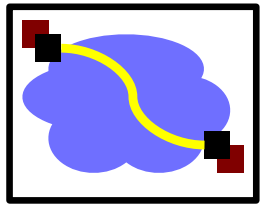


Network Service Model



- What is the *service model* for inter-network*?*
  - Defines what promises that the network gives for any transmission
  - Defines what type of failures to expect
- Ethernet/Internet: *best-effort* – packets can get lost, etc.

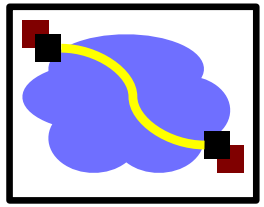
Possible Failure models



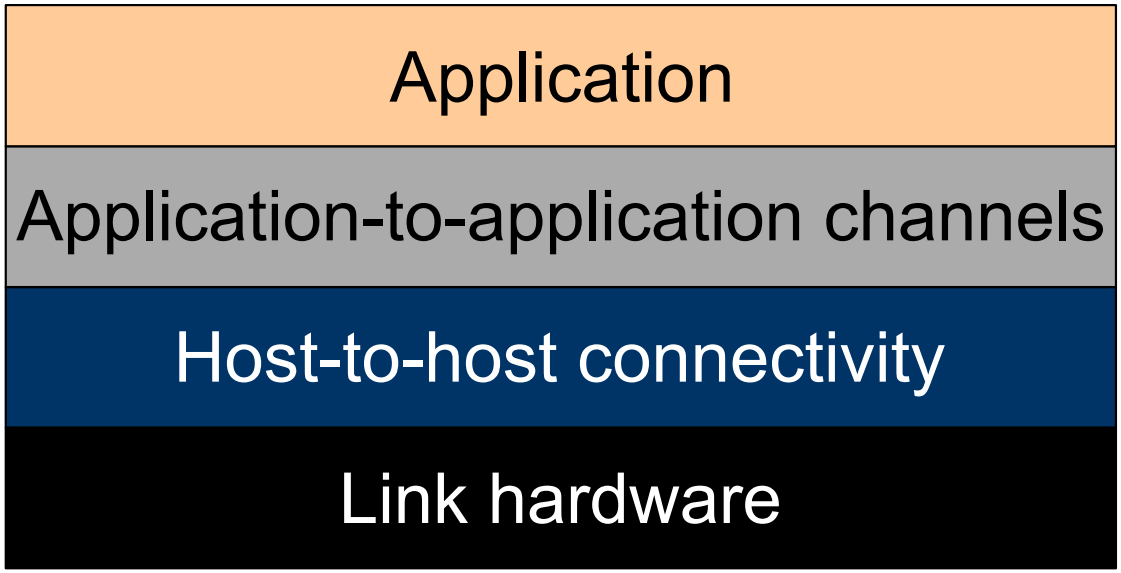
- Fail-stop:
  - When something goes wrong, the process stops / crashes / etc.
- Fail-slow or fail-stutter:
  - Performance may vary on failures as well
- Byzantine:
  - Anything that can go wrong, will.
  - Including malicious entities taking over your computers and making them do whatever they want.
- These models are useful for proving things;
- The real world typically has a bit of everything.

- Deciding which model to use is important!

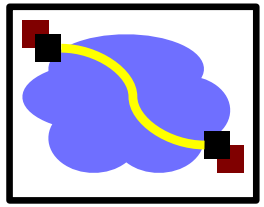
What is Layering?



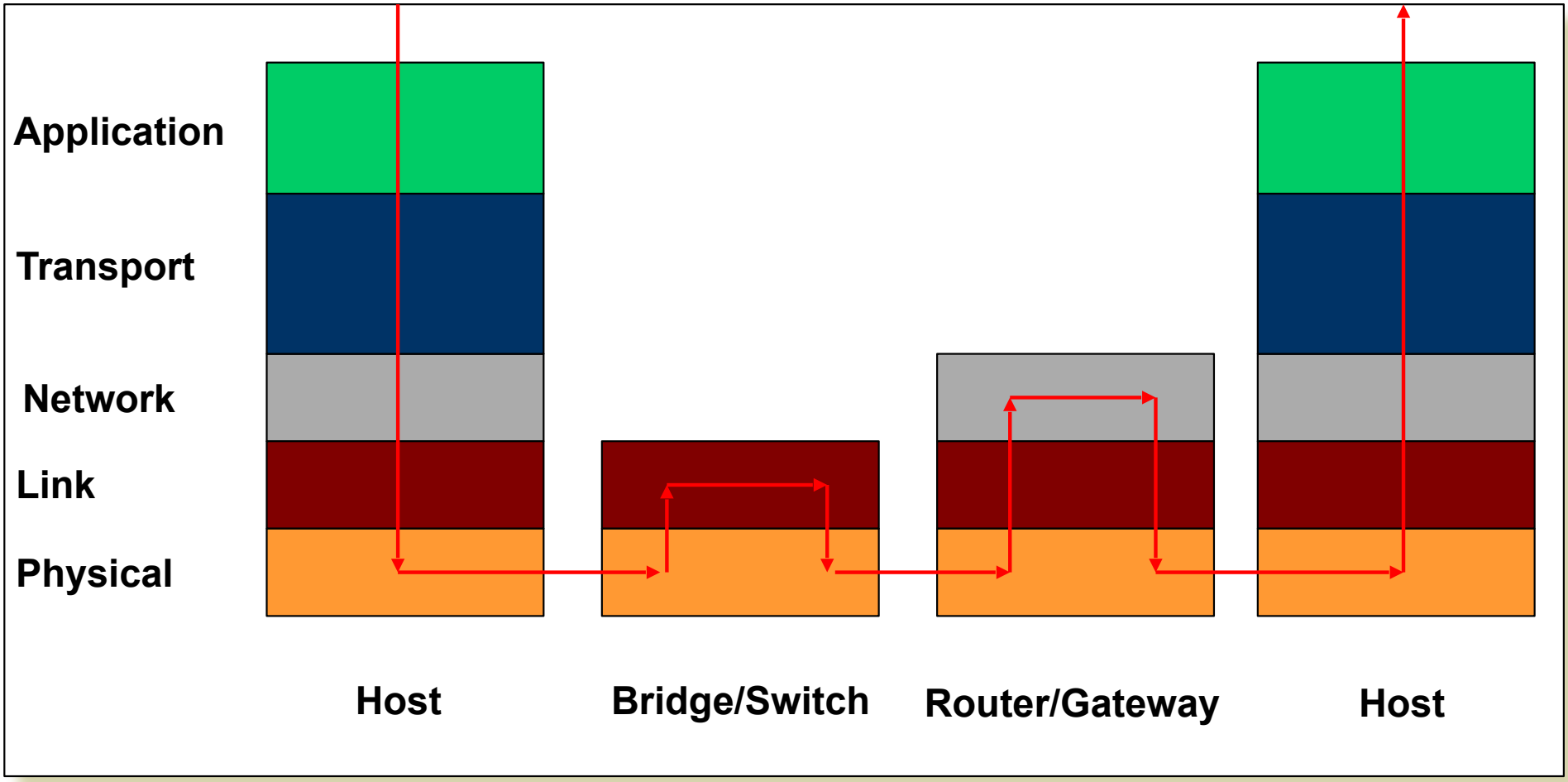
- Modular approach to network functionality
- Example:



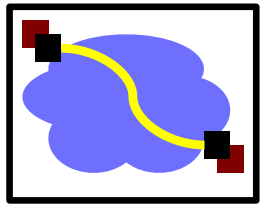
IP Layering



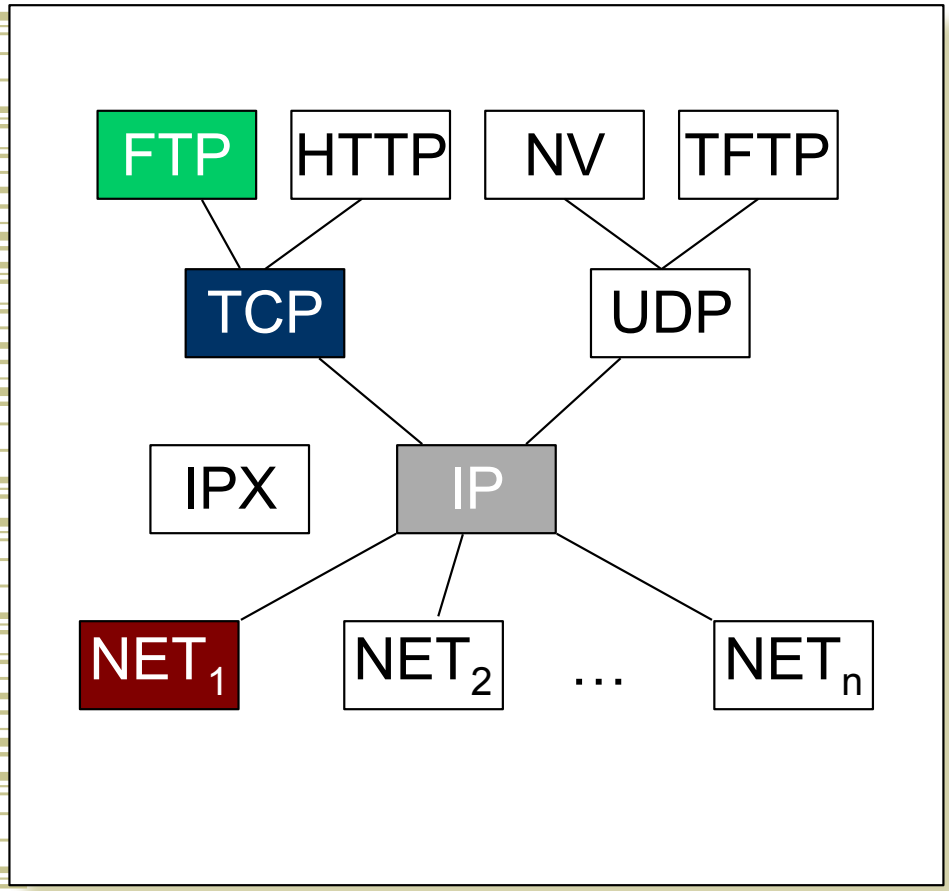
- Relatively simple

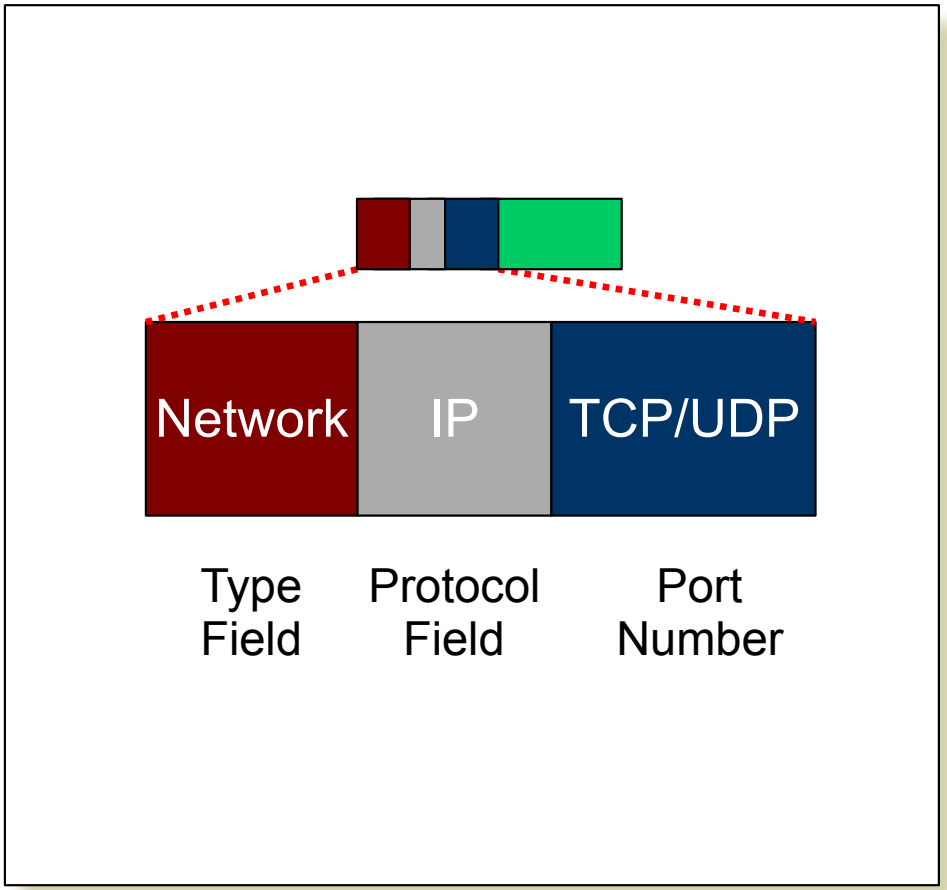


Protocol Demultiplexing

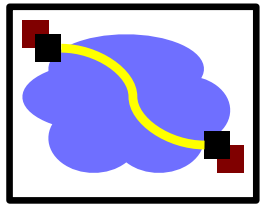


- Multiple choices at each layer





Goals [Clark88]



0 **Connect existing networks**

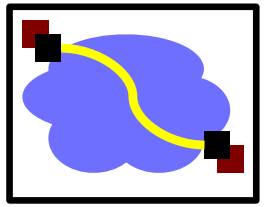
initially ARPANET and ARPA packet radio network

1. Survivability

ensure communication service even in the presence of network and router failures

2. Support multiple types of services
3. Must accommodate a variety of networks
4. Allow distributed management
5. Allow host attachment with a low level of effort
6. Be cost effective
7. Allow resource accountability

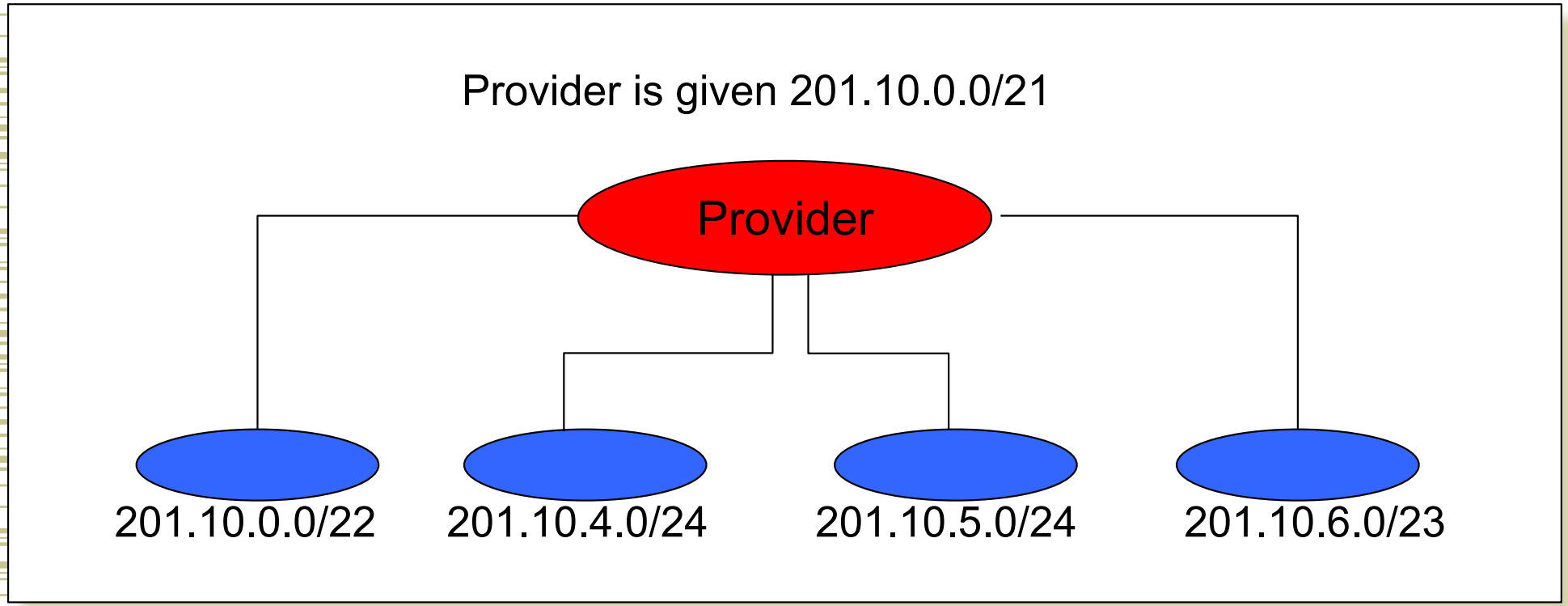
Goal 1: Survivability



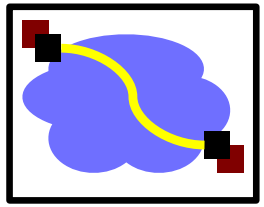
- If network is disrupted and reconfigured…
  - Communicating entities should not care!
  - No higher-level state reconfiguration
- How to achieve such reliability?
  - Where can communication state be stored?

	Network	Host
Failure handing	Replication	“Fate sharing”
Net Engineering	Tough	Simple
Switches	Maintain state	Stateless
Host trust	Less	More

CIDR IP Address Allocation

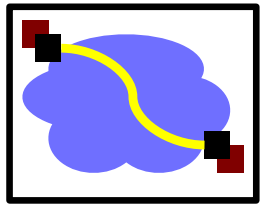


Ethernet Frame Structure (cont.)



- Addresses:
  - 6 bytes
  - Each adapter is given a globally unique address at manufacturing time
    - Address space is allocated to manufacturers
      - 24 bits identify manufacturer
      - E.g., 0:0:15:* → 3com adapter
    - Frame is received by all adapters on a LAN and dropped if address does not match
  - Special addresses
    - Broadcast – FF:FF:FF:FF:FF:FF is “everybody”
    - Range of addresses allocated to multicast
      - Adapter maintains list of multicast groups node is interested in

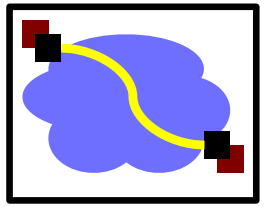
User Datagram Protocol (UDP): An Analogy



UDP	Postal Mail
• Single socket to receive messages • No guarantee of delivery • Not necessarily in-order delivery • Datagram – independent packets • Must address each packet	• Single mailbox to receive letters • Unreliable ☺ • Not necessarily in-order delivery • Letters sent independently • Must address each letter

Example UDP applications
Multimedia, voice over IP

Transmission Control Protocol (TCP): An Analogy



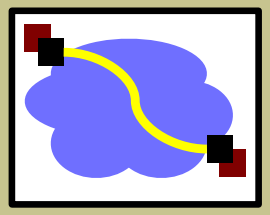
TCP

- Reliable – guarantee delivery
- Byte stream – in-order delivery
- Connection-oriented – single socket per connection
- Setup connection followed by data transfer

Telephone Call

- Guaranteed delivery
- In-order delivery
- Connection-oriented
- Setup connection followed by conversation

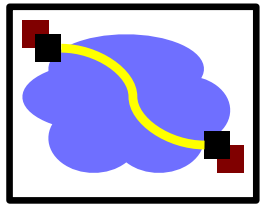
Example TCP applications
Web, Email, Telnet



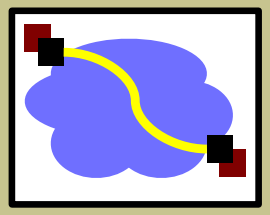
15-440 Distributed Systems

Lecture 5 – Classical Synchronization

Classic synchronization primitives



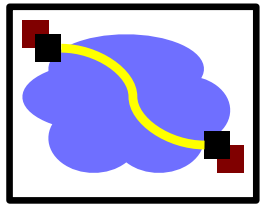
- Basics of concurrency
  - Correctness (achieves Mutex, no deadlock, no livelock)
  - Efficiency, no spinlocks or wasted resources
  - Fairness
- Synchronization mechanisms
  - Semaphores (P() and V() operations)
  - Mutex (binary semaphore)
  - Condition Variables (allows a thread to sleep)
    - Must be accompanied by a mutex
    - Wait and Signal operations
  - Work through examples again + GO primitives



15-440 Distributed Systems

Lecture 6 – RPC

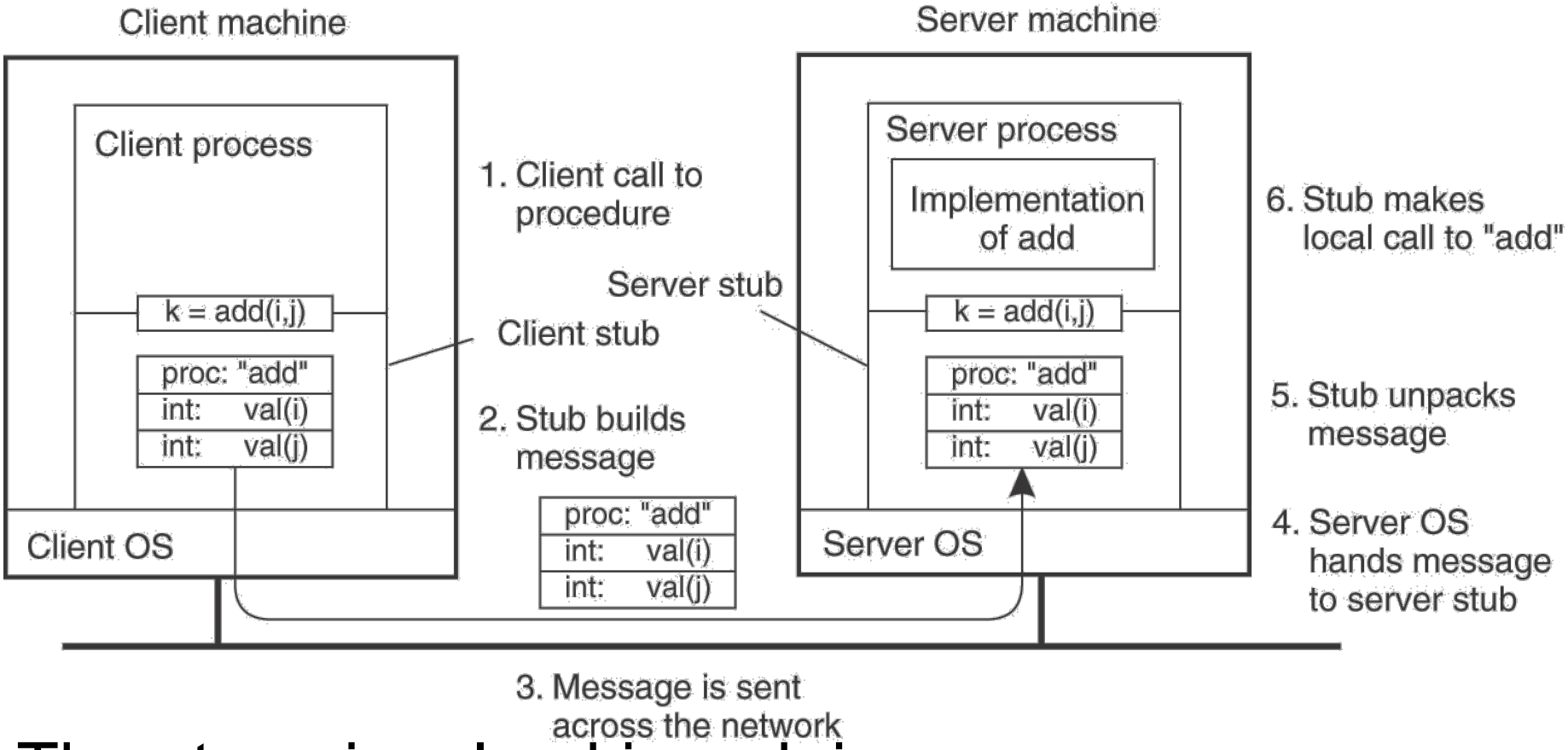
RPC Goals



- Ease of programming
- Hide complexity
- Automates task of implementing distributed computation
- Familiar model for programmers (just make a function call)

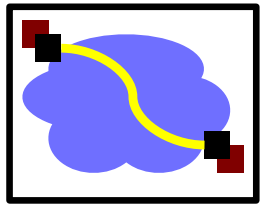
Historical note: Seems obvious in retrospect, but RPC was only invented in the '80s. See Birrell & Nelson, "Implementing Remote Procedure Call" ... or
Bruce Nelson, Ph.D. Thesis, Carnegie Mellon University: Remote Procedure Call., 1981 :)

Passing Value Parameters (1)



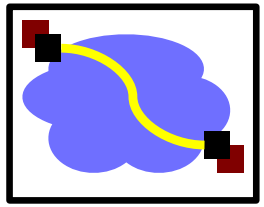
- The steps involved in a doing a remote computation through RPC.

Stubs: obtaining transparency



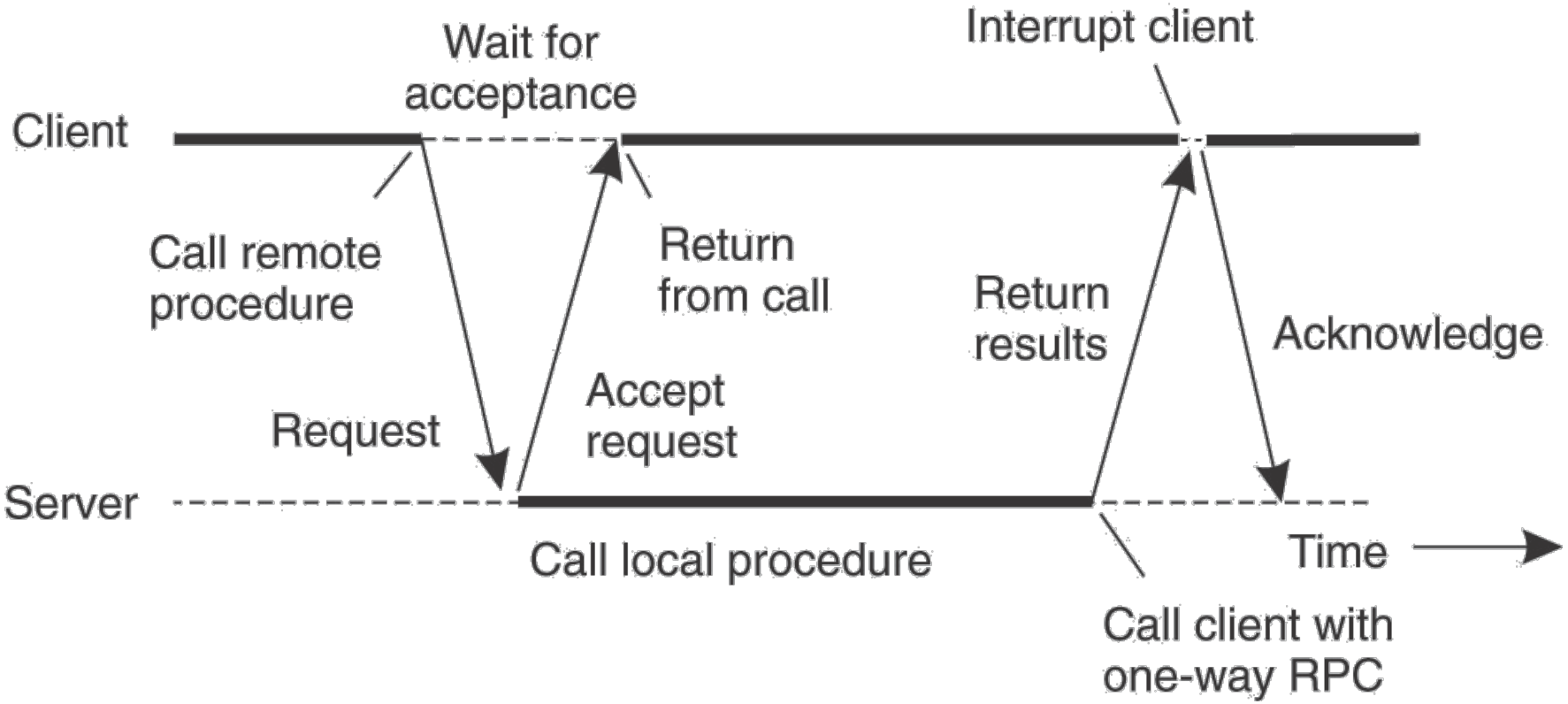
- Compiler generates from API stubs for a procedure on the client and server
- Client stub
  - **Marshals** arguments into machine-independent format
  - Sends request to server
  - Waits for response
  - **Unmarshals** result and returns to caller
- Server stub
  - **Unmarshals** arguments and builds stack frame
  - Calls procedure
  - Server stub **marshals** results and sends reply

Real solution: break transparency



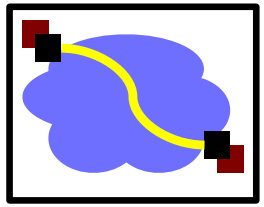
- Possible semantics for RPC:
  - Exactly-once
    - Impossible in practice
  - At least once:
    - Only for idempotent operations
  - At most once
    - Zero, don't know, or once
  - Zero or once
    - Transactional semantics

Asynchronous RPC (3)

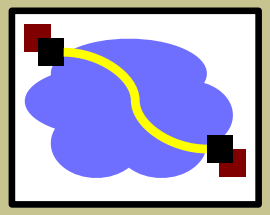


- A client and server interacting through two asynchronous RPCs.

Important Lessons



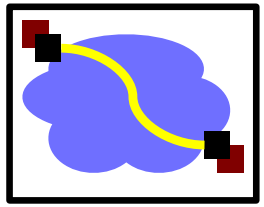
- Procedure calls
  - Simple way to pass control and data
  - Elegant transparent way to distribute application
  - Not only way…
- Hard to provide true transparency
  - Failures
  - Performance
  - Memory access
  - Etc.
- How to deal with hard problem → give up and let programmer deal with it
  - "Worse is better"



15-440 Distributed Systems

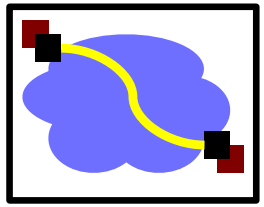
Lecture 7,8 – Distributed File Systems

Why DFSs?



- Why Distributed File Systems:
  - Data sharing among multiple users
  - User mobility
  - Location transparency
  - Backups and centralized management
  - Examples: NFS (v1 – v4), AFS, CODA, LBFS
- Idea: Provide file system interfaces to remote FS's
  - Challenge: heterogeneity, scale, security, concurrency,..
  - Non-Challenges: AFS meant for campus community
  - Virtual File Systems: pluggable file systems
  - Use RPC's

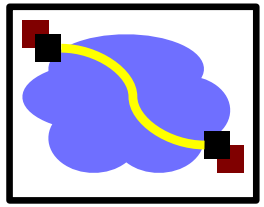
NFS vs AFS Goals



- AFS: Global distributed file system
  - “One AFS”, like “one Internet”
  - **LARGE** numbers of clients, servers (1000’s cache files)
  - Global namespace (organized as cells) => location transparency
  - Clients w/disks => cache, Write sharing rare, callbacks
  - Open-to-close consistency (session semantics)

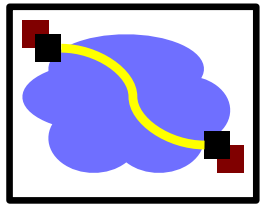
- NFS: Very popular Network File System
  - NFSv4 meant for wide area
  - Naming: per-client view (/home/yuvraj/…)
  - Cache data in memory, not on disk, write through cache
  - Consistency model: buffer data (eventual, ~30seconds)
  - Requires significant resounces as users scale

Session Semantics in AFS v2



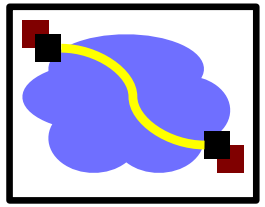
- What it means:
  - A file write is visible to processes on the same box immediately, but not visible to processes on other machines until the file is closed
  - When a file is closed, changes are visible to new opens, but are not visible to “old” opens
  - All other file operations are visible everywhere immediately
- Implementation
  - Dirty data are buffered at the client machine until file close, then flushed back to server, which leads the server to send “break callback” to other clients

DFS Important Bits (2)



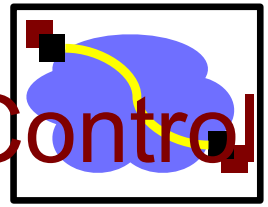
- Client-side caching is a fundamental technique to improve scalability and performance
  - But raises important questions of cache consistency
- Timeouts and callbacks are common methods for providing (some forms of) consistency.
- AFS picked close-to-open consistency as a good balance of usability (the model seems intuitive to users), performance, etc.
  - AFS authors argued that apps with highly concurrent, shared access, like databases, needed a different model

DFS: Name-Space Construction and Organization



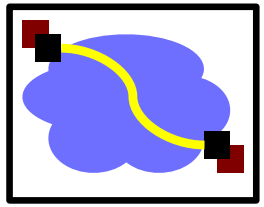
- NFS: per-client linkage
  - Server: export /root/fs1/
  - Client: mount server:/root/fs1 /fs1
- AFS: global name space
  - Name space is organized into Volumes
    - Global directory /afs;
    - /afs/cs.wisc.edu/vol1/…; /afs/cs.stanford.edu/vol1/…
  - Each file is identified as fid = <vol_id, vnode #, unique identifier>
  - All AFS servers keep a copy of “volume location database”, which is a table of vol_id→ server_ip mappings

DFS: User Authentication and Access Control

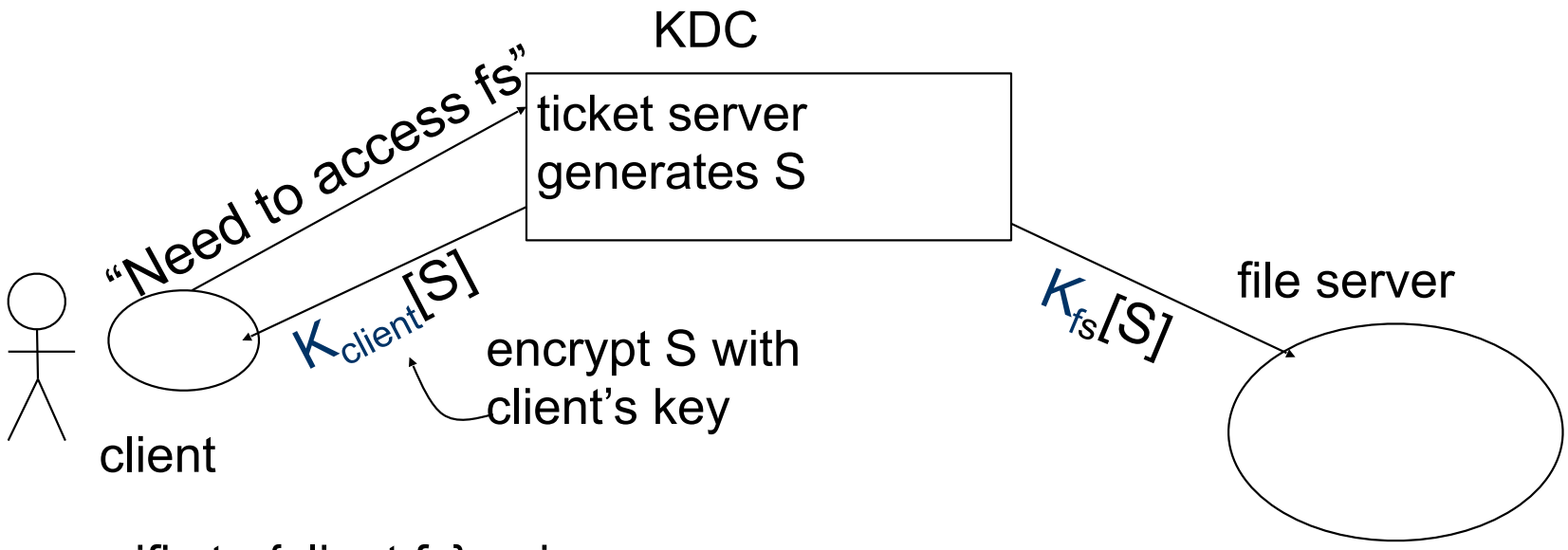


- User X logs onto workstation A, wants to access files on server B
  - How does A tell B who X is?
  - Should B believe A?
- Choices made in NFS V2
  - All servers and all client workstations share the same <uid, gid> name space → B send X's <uid,gid> to A
    - Problem: root access on any client workstation can lead to creation of users of arbitrary <uid, gid>
  - Server believes client workstation unconditionally
    - Problem: if any client workstation is broken into, the protection of data on the server is lost;
    - <uid, gid> sent in clear-text over wire → request packets can be faked easily

A Better AAA System: Kerberos

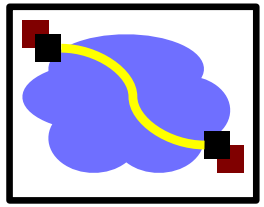


- Basic idea: shared secrets
  - User proves to KDC who he is; KDC generates shared secret between client and file server



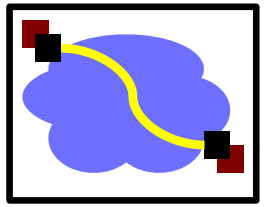
S: specific to {client,fs} pair;
"short-term session-key"; expiration time (e.g. 8 hours)

Coda Summary



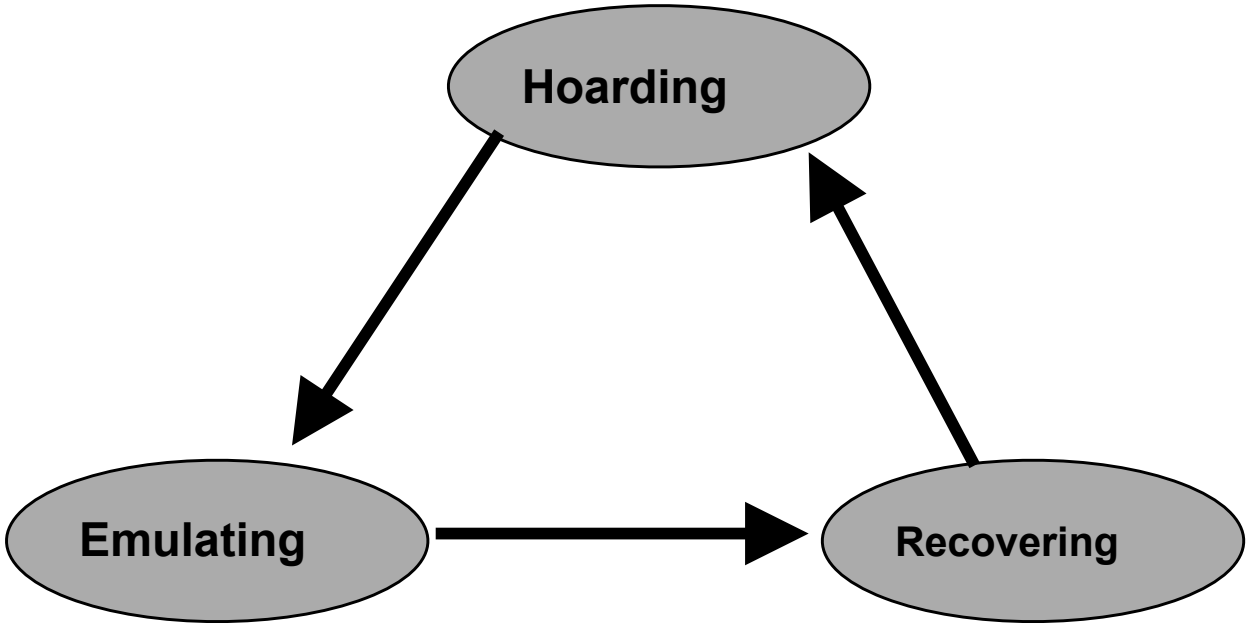
- Distributed File System built for mobility
  - Disconnected operation key idea
- Puts scalability and availability before data consistency
  - Unlike NFS
- Assumes that inconsistent updates are very infrequent
- Introduced disconnected operation mode and file hoarding and the idea of “reintegration”

CODA: Design Rationale



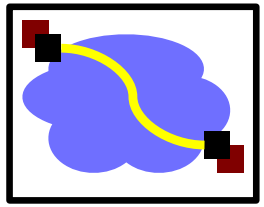
- Scalability
  - Callback cache coherence (inherit from AFS)
  - Whole file caching
  - Fat clients. (security, integrity)
  - Avoid system-wide rapid change
- Portable workstations
  - User's assistance in cache management

Coda States



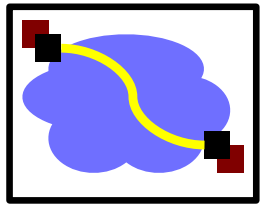
1. ***Hoarding:***
   Normal operation mode
2. ***Emulating:***
   Disconnected operation mode
3. ***Reintegrating:***
   Propagates changes and detects inconsistencies

Hoarding



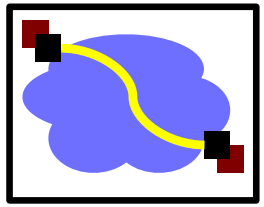
- Hoard useful data for disconnection
- Balance the needs of connected and disconnected operation.
  - Cache size is restricted
  - Unpredictable disconnections
- Uses user specified preferences + usage patterns to decide on files to keep in hoard

Emulation



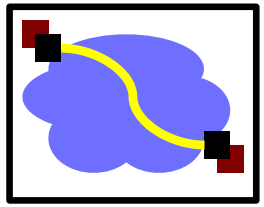
- In emulation mode:
  - Attempts to access files that are not in the client caches appear as failures to application
  - All changes are written in a persistent log, the client modification log (CML)
  - Coda removes from log all obsolete entries like those pertaining to files that have been deleted

Reintegration



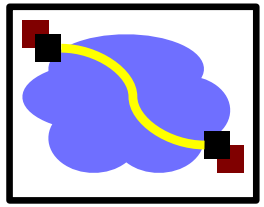
- When workstation gets reconnected, Coda initiates a ***reintegration process***
  - Performed one volume at a time
  - Venus ships replay log to all volumes
  - Each volume performs a log replay algorithm
- Only care about write/write confliction
  - Conflict resolution succeeds?
    - Yes. Free logs, keep going…
    - No. Save logs to a tar. Ask for help
- In practice:
  - No Conflict at all! Why?
  - Over 99% modification by the same person
  - Two users modify the same obj within a day: <0.75%

Low Bandwidth File System Key Ideas



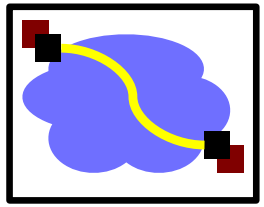
- A network file systems for slow or wide-area networks
- Exploits similarities between files
  - Avoids sending data that can be found in the server's file system or the client's cache
  - Uses RABIN fingerprints on file content (file chunks)
    - Can deal with byte offsets when part of file change
- Also uses conventional compression and caching
- Requires 90% less bandwidth than traditional network file systems

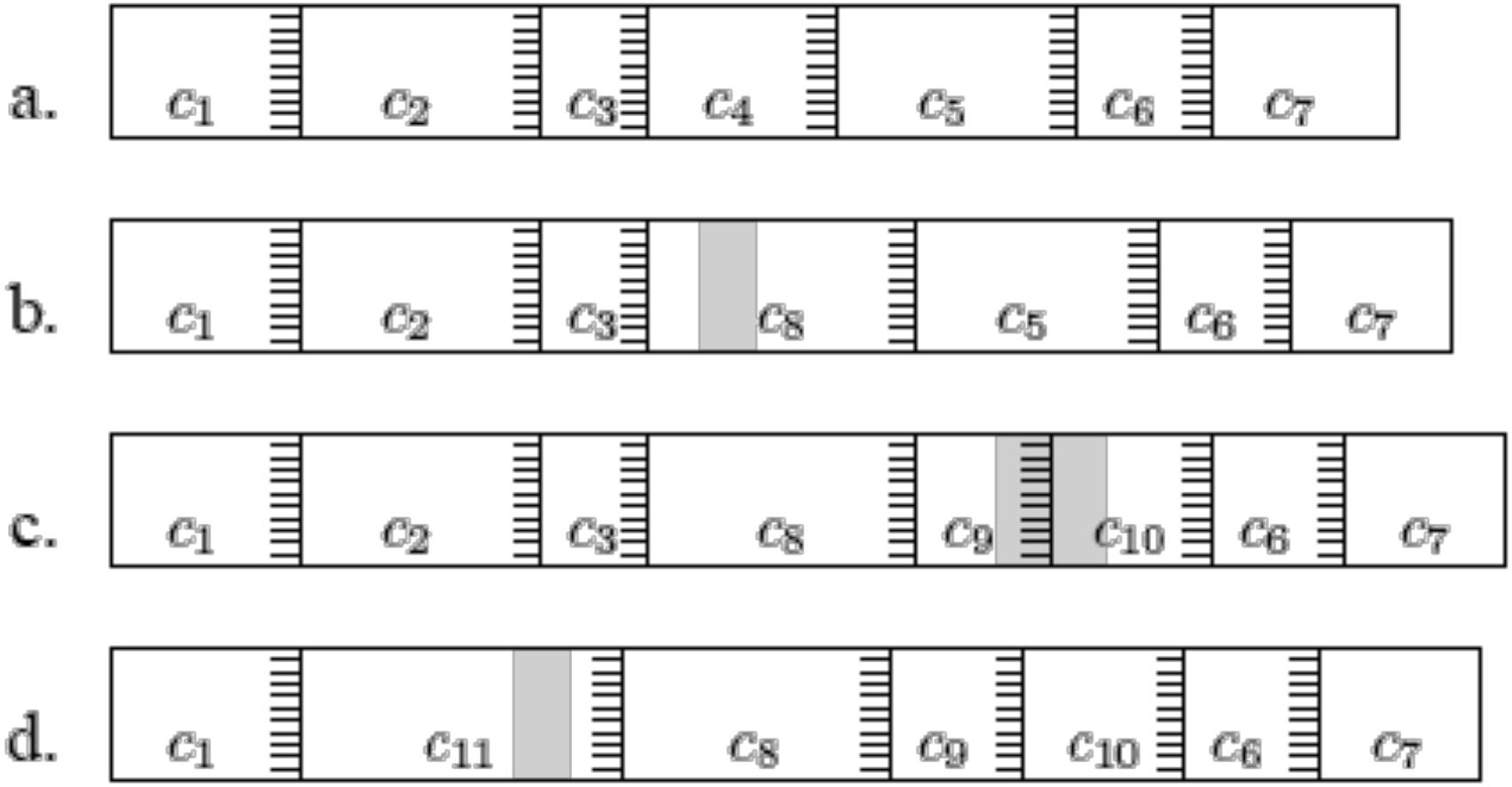
LBFS chunking solution



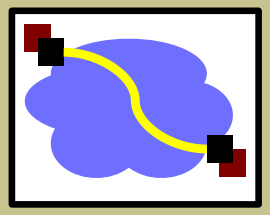
- Considers only non-overlapping chunks
- Sets chunk boundaries based on file contents rather than on position within a file
- Examines every overlapping 48-byte region of file to select the boundary regions called *breakpoints* using Rabin fingerprints
  - When low-order 13 bits of region's fingerprint equals a chosen value, the region constitutes a breakpoint

Effects of edits on file chunks





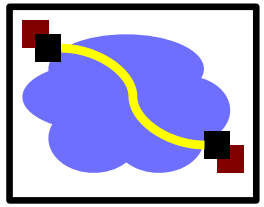
- Chunks of file before/after edits
  - Grey shading show edits
- Stripes show regions with magic values that create chunk boundaries

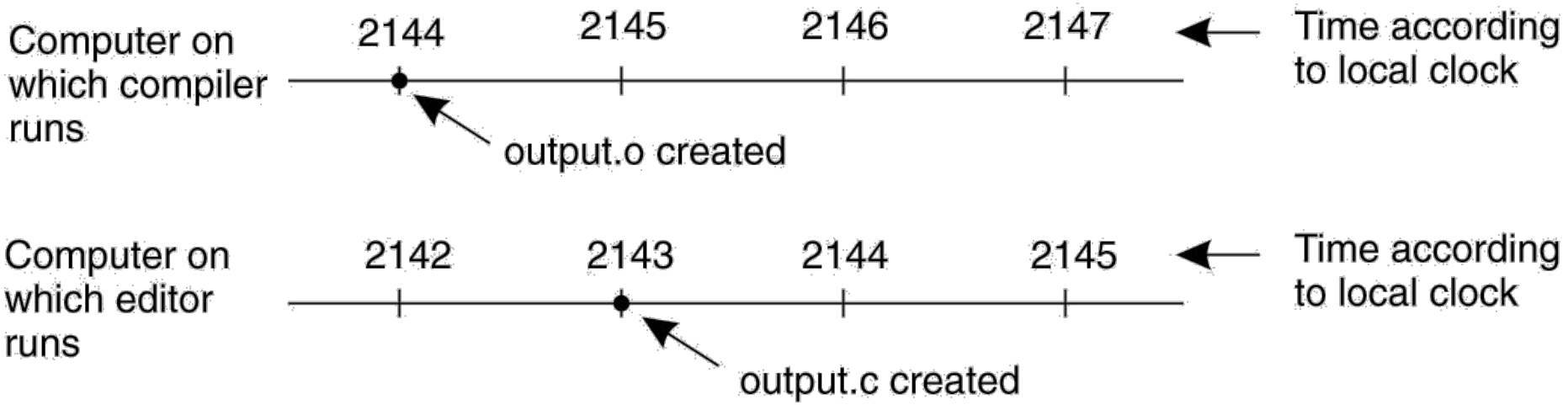


15-440 Distributed Systems

Lecture 9 – Time Synchronization

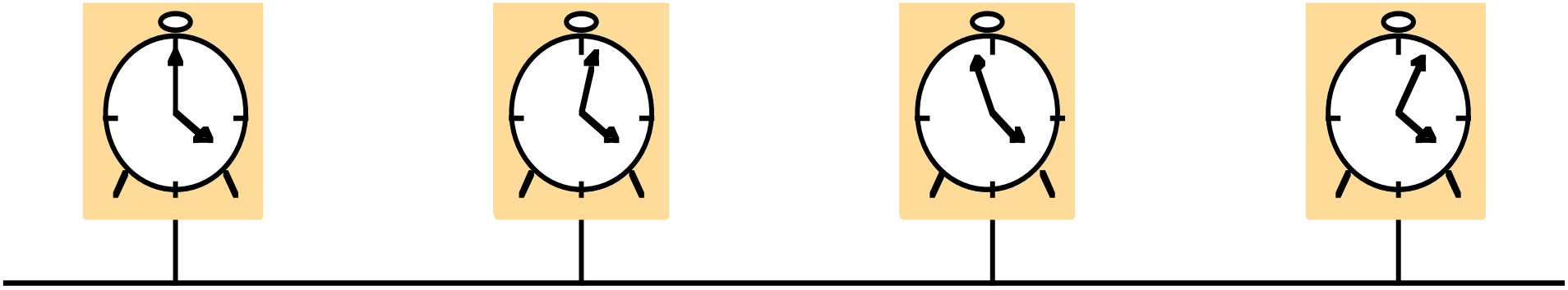
Impact of Clock Synchronization





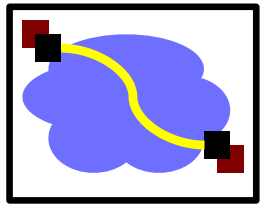
- When each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time.

Clocks in a Distributed System

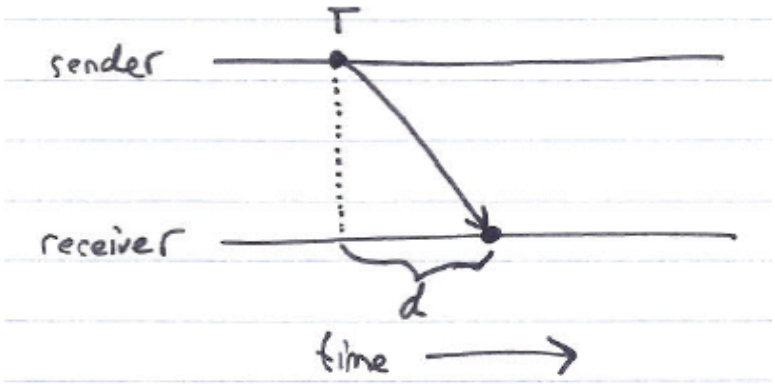


- Computer clocks are not generally in perfect agreement
  - **Skew**: the difference between the times on two clocks (at any instant)
- Computer clocks are subject to clock drift (they count time at different rates)
  - **Clock drift rate**: the difference per unit of time from some ideal reference clock
  - Ordinary quartz clocks drift by about 1 sec in 11-12 days. (10^{-6} secs/sec).
  - High precision quartz clocks drift rate is about 10^{-7} or 10^{-8} secs/sec

Perfect networks

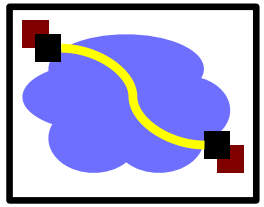


- Messages always arrive, with propagation delay exactly *d*

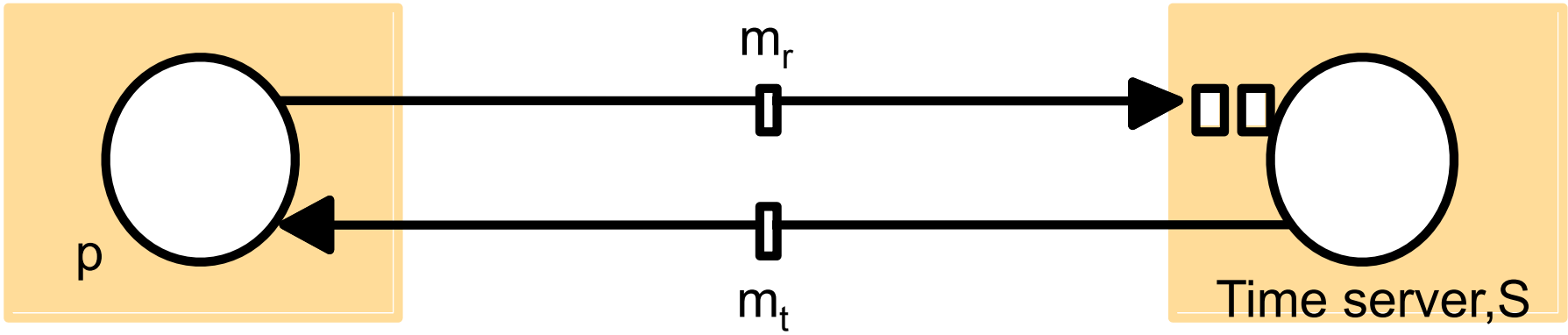


- Sender sends time *T* in a message
- Receiver sets clock to *T*+*d*
  - Synchronization is exact

Cristian's Time Sync

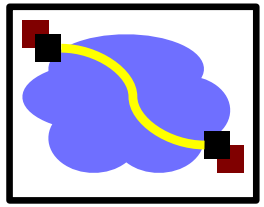


- A time server *S* receives signals from a UTC source
  - Process *p* requests time in m_r and receives *t* in m_t from *S*
  - *p* sets its clock to *t* + *RTT*/2
  - Accuracy ± (*RTT*/2 - *min*) :
    - because the earliest time *S* puts *t* in message m_t is *min* after *p* sent m_r.
    - the latest time was *min* before m_t arrived at *p*
    - the time by *S*'s clock when m_t arrives is in the range [*t*+*min*, *t* + *RTT* - *min*]



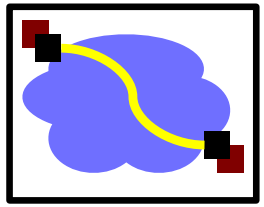
T_{round} is the round trip time recorded by *p*
min is an estimated minimum round trip time

Berkeley algorithm

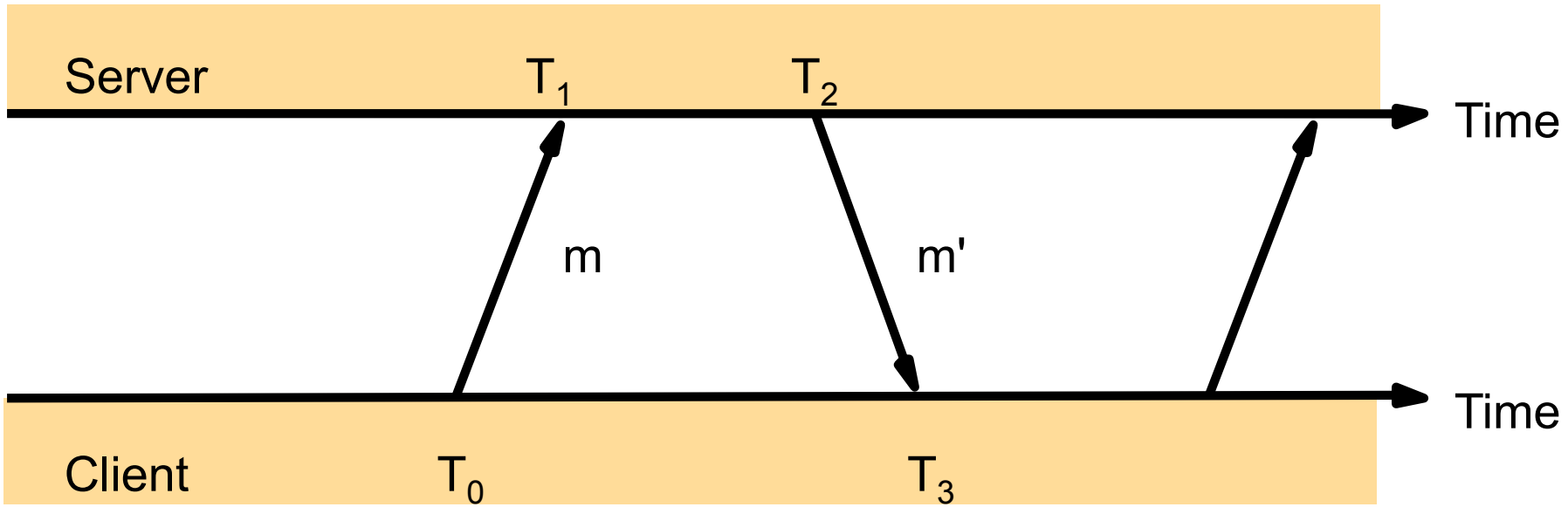


- Cristian's algorithm -
  - a single time server might fail, so they suggest the use of a group of synchronized servers
  - it does not deal with faulty servers
- Berkeley algorithm (also 1989)
  - An algorithm for internal synchronization of a group of computers
  - A *master* polls to collect clock values from the others (*slaves*)
  - The master uses round trip times to estimate the slaves' clock values
  - It takes an average (eliminating any above average round trip time or with faulty clocks)
  - It sends the required adjustment to the slaves (better than sending the time which depends on the round trip time)
  - Measurements
    - 15 computers, clock synchronization 20-25 millisecs drift rate < 2×10^{-5}
    - If master fails, can elect a new master to take over (not in bounded time)

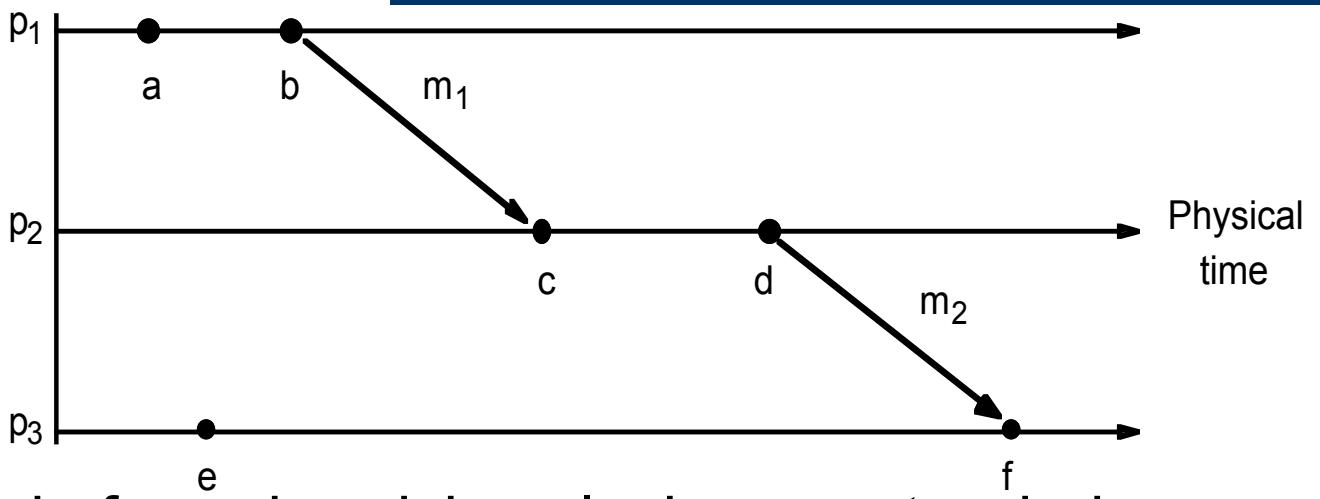
NTP Protocol



- All modes use UDP
- Each message bears timestamps of recent events:
  - Local times of Send and Receive of previous message
  - Local times of Send of current message
- Recipient notes the time of receipt T_3 (we have T_0, T_1, T_2, T_3)



Logical time and logical clocks (Lamport 1978)

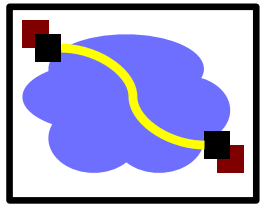


- Instead of synchronizing clocks, event ordering can be used

  1. If two events occurred at the same process p_i (i = 1, 2, … N) then they occurred in the order observed by p_i, that is the definition of: "$\rightarrow_i$"
  2. when a message, m is sent between two processes, send(m) happens before receive(m)
  3. The happened before relation is transitive

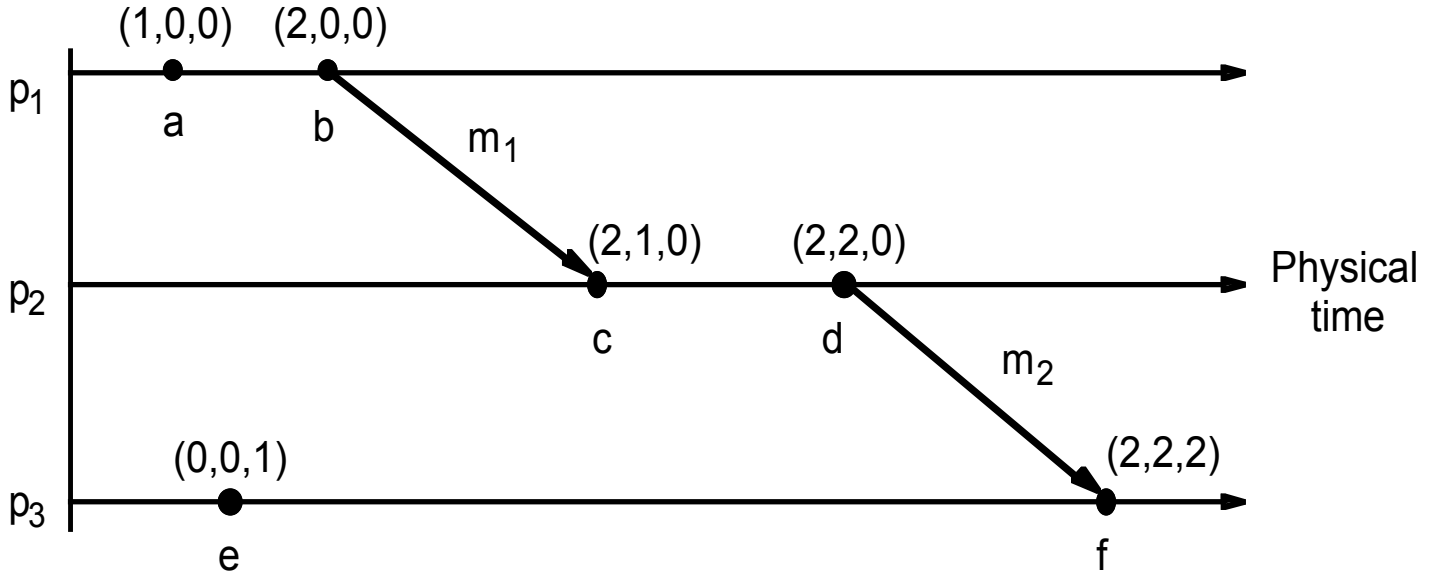
- The happened before relation is the relation of causal ordering

Total-order Lamport clocks

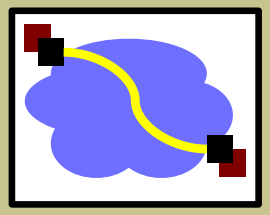


- Many systems require a total-ordering of events, not a partial-ordering
- Use Lamport's algorithm, but break ties using the process ID
  - $L(e) = M * L_i(e) + i$
    - M = maximum number of processes
    - i = process ID

Vector Clocks



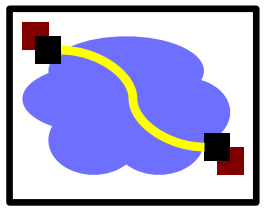
- Note that e → e' implies V(e)<V(e'). The converse is also true
- Can you see a pair of parallel events?
  - *c* || *e* (parallel) because neither *V*(c) <= *V*(*e*) nor *V*(*e*) <= *V*(*c*)

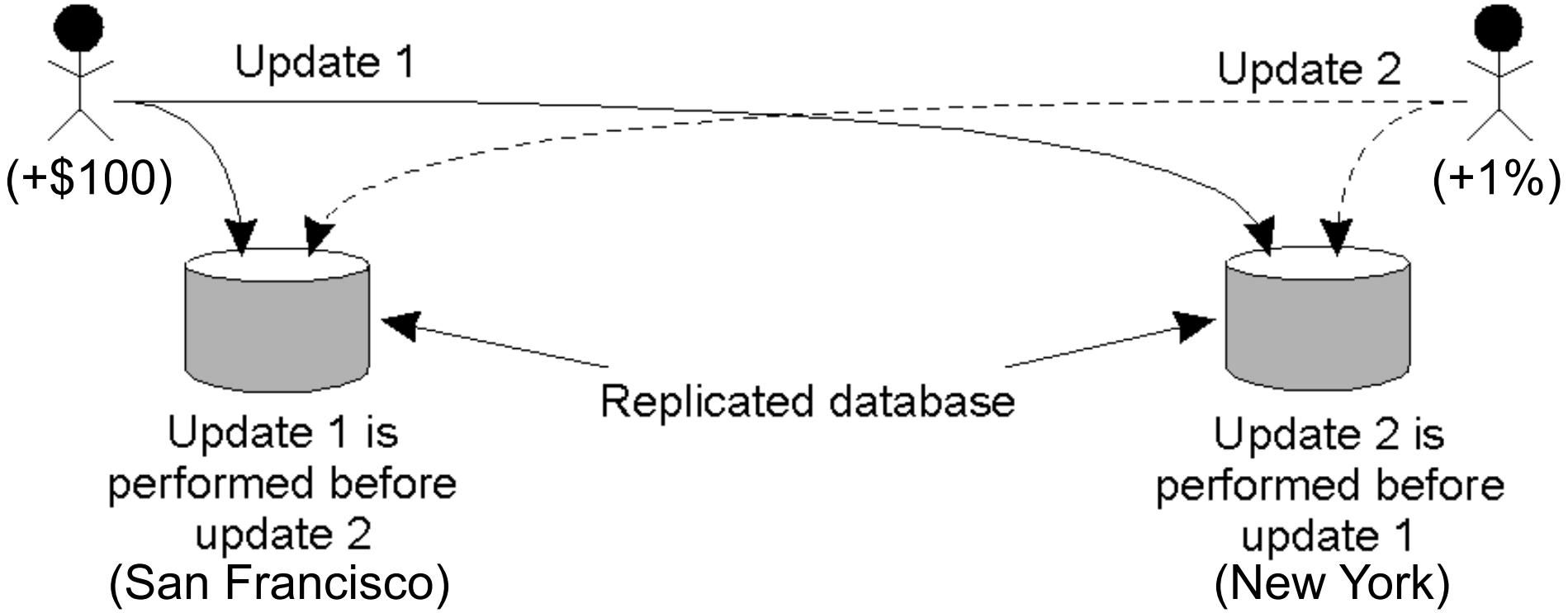


15-440 Distributed Systems

Lecture 10 – Mutual Exclusion

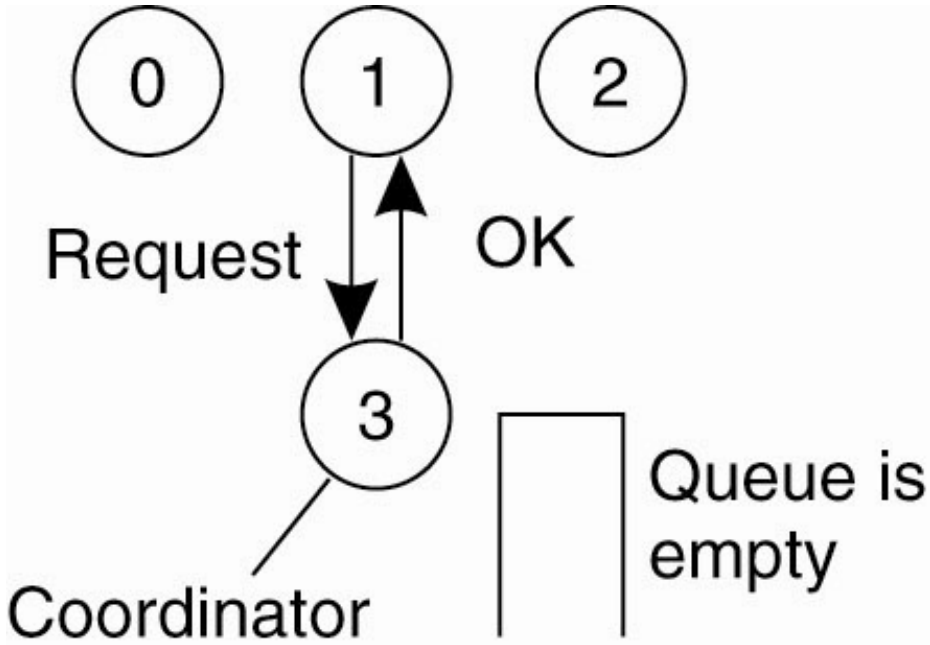
Example: Totally-Ordered Multicasting





- San Fran customer adds $100, NY bank adds 1% interest
  - San Fran will have $1,111 and NY will have $1,110
- Updating a replicated database and leaving it in an inconsistent state.
- Can use Lamport's to totally order

Mutual Exclusion
A Centralized Algorithm (1)



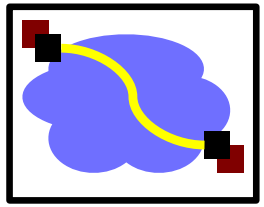
@ Server:

```
while true:
    m = Receive()
    If m == (Request, i):
If Available():
            Send (Grant) to i
```

@ Client → Acquire:

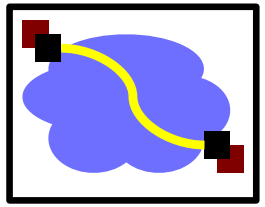
```
Send (Request, i) to coordinator
Wait for reply
```

Distributed Algorithm Strawman



- Assume that there are n coordinators
  - Access requires a majority vote from m > n/2 coordinators.
  - A coordinator always responds immediately to a request with GRANT or DENY
- Node failures are still a problem
- Large numbers of nodes requesting access can affect availability

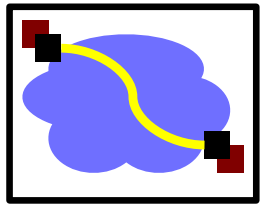
A Distributed Algorithm 2 (Lamport Mutual Exclusion)



- Every process maintains a queue of pending requests for entering critical section in order. The queues are ordered by virtual time stamps derived from Lamport timestamps
  - For any events e, e' such that e --> e' (causality ordering), T(e) < T(e')
  - For any distinct events e, e', T(e) != T(e')

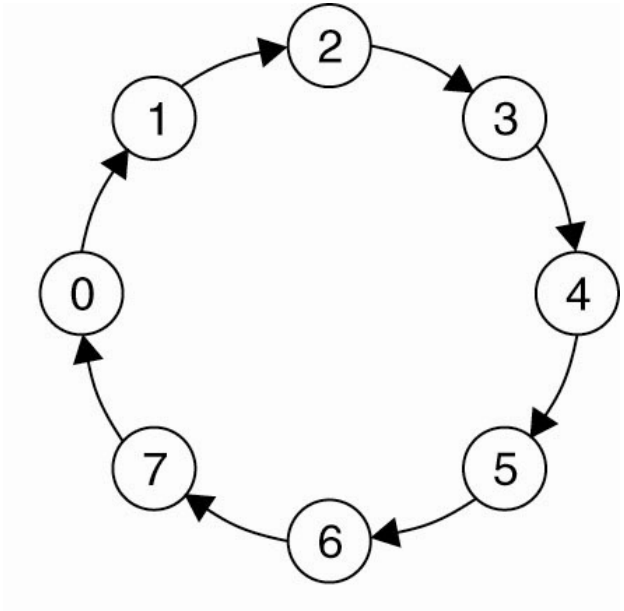
- When node i wants to enter C.S., it sends time-stamped request to all other nodes (including itself)
  - Wait for replies from all other nodes.
  - If own request is at the head of its queue and all replies have been received, enter C.S.
  - Upon exiting C.S., remove its request from the queue and send a release message to every process.

A Distributed Algorithm 3 (Ricart & Agrawala)



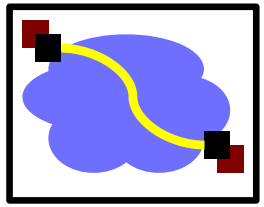
- Also relies on Lamport totally ordered clocks.
- When node i wants to enter C.S., it sends time-stamped request to all other nodes. These other nodes reply (eventually). When i receives n-1 replies, then can enter C.S.
- Trick: Node j having earlier request doesn't reply to i until after it has completed its C.S.

A Token Ring Algorithm



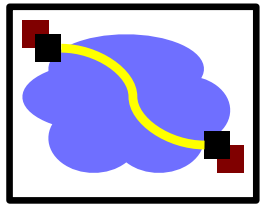
- Organize the processes involved into a logical ring
- One token at any time → passed from node to node along ring

A Token Ring Algorithm

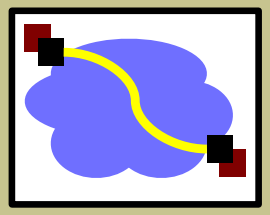


- Correctness:
  - Clearly safe: Only one process can hold token
- Fairness:
  - Will pass around ring at most once before getting access.
- Performance:
  - Each cycle requires between 1 - ∞ messages
  - Latency of protocol between 0 & n-1
- Issues
  - Lost token

Summary



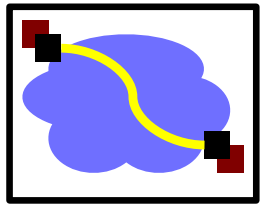
- Lamport algorithm demonstrates how distributed processes can maintain consistent replicas of a data structure (the priority queue).
- Ricart & Agrawala's algorithms demonstrate utility of logical clocks.
- Centralized & ring based algorithms much lower message counts
- None of these algorithms can tolerate failed processes or dropped messages.



15-440 Distributed Systems

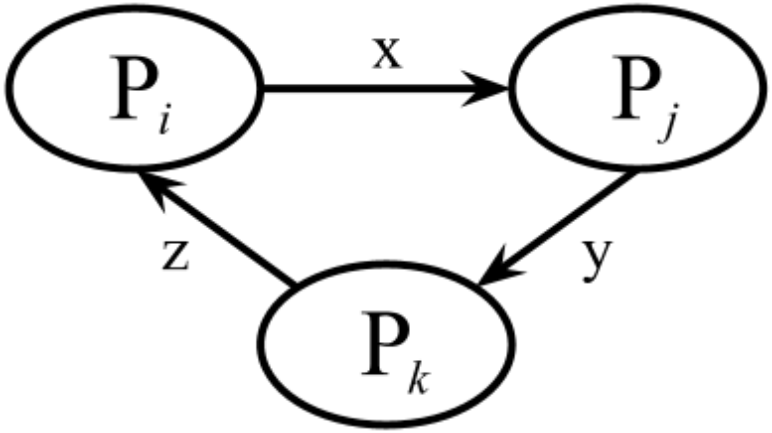
Lecture 09 – Distributed Concurrency Management
Tuesday Sept 29th, 2016

Distributed Concurrency Management



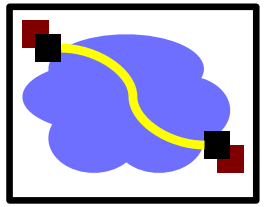
- Multiple-Objects, Multiple Servers. Ignore Failure
  - Single Server: Transactions (RD/WR to Global State)
  - ACID: Atomicity, Consistency, Isolation, Durability
  - Learn what these mean in the context of transactions
  - E.g. banking app => ACID is violated if not careful
  - Solutions: 2-phase locking (General, strict, strong strict)
    - Deadling with deadlocks => build “waits-for” graph
    - Transactions: 2 phases (prep, commit/abort)
      - Preparation: generate Lock Set “L”, Updates “U”
      - COMMIT (updated global state), ABORT (leave state as is)
      - Example using banking app

2-Phase Locking



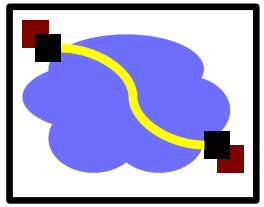
- Building a “Wait-for’ graph for state of locks. Vertices represent transactions. Edge from vertex i to vertex j if transaction i is waiting for lock held by transaction j.
- In this case, what happens? => Cycle is a deadlock
- Label edges with its lock ID. For any cycle, there must be some pair of edges (i, j), (j, k) labeled with values x & y such that x > y. That implies that transaction j is holding lock x and it wants lock y, where x > y. That implies that j is not acquiring its lock in proper order.
- General scheme called 2-phase locking
  - More precisely: strong strict two phase locking

2-Phase Locking



- General 2-phase locking
  - Phase 1: Acquire or Escalate Locks (e.g. read => write)
  - Phase 2: Release or de-escalate lock
- Strict 2-phase locking
  - Phase 1: (same as before)
  - Phase 2: Release WRITE lock at end of transaction only
- Strong Strict 2-phase locking
  - Phase 2: Release ALL locks at end of transaction only.
  - Most common version, required for ACID properties

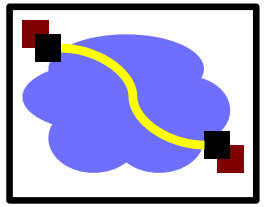
Transactions – split into 2 phases



- Phase 1: Preparation:
  - Determine what has to be done, how it will change state, without actually altering it.
  - Generate Lock set “L”
  - Generate List of Updates “U”

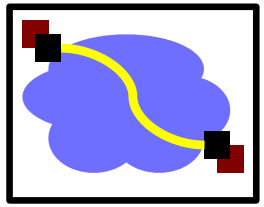
- Phase 2: Commit or Abort
  - Everything OK, then update global state
  - Transaction cannot be completed, leave global state as is
  - In either case, RELEASE ALL LOCKS

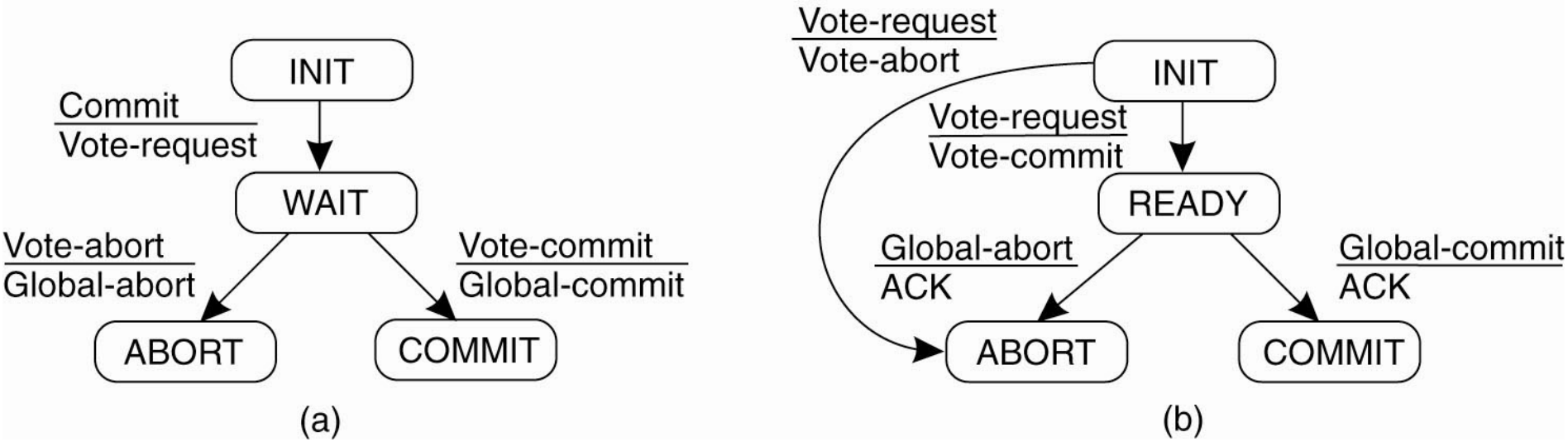
Distributed Transactions – 2PC



- Similar idea as before, but:
  - State spread across servers (maybe even WAN)
  - Want to enable single transactions to read and update global state while maintaining ACID properties
- Overall Idea:
  - Client initiate transaction. Makes use of “co-ordinator”
  - All other relevant servers operate as “participants”
  - Co-ordinator assigns unique transaction ID (TID)
- 2 Phase-Commit
  - Prepare & Vote (client determine states, talk to Coord.)
  - Commit/Abort Phase (co-ordinator boardcast to clients)

Two-Phase Commit (1)

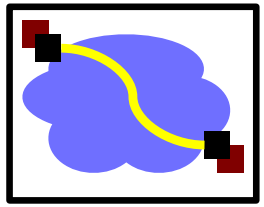




(a) The finite state machine for the coordinator in 2PC.
(b) The finite state machine for a participant.

- Coordinator/Participant can be blocked in 3 states:
  - Participant: Waiting in INIT state for VOTE_REQUEST
  - Coordinator: Blocked in WAIT state, listening for votes
  - Participant: blocked in READY state, waiting for global vote

Two-Phase Commit (2)

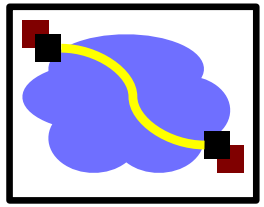


- What if a “READY” participant does not receive the global commit? Can’t just abort => figure out what message a co-ordinator may have sent.
- Approach: ask other partcipants
  - Take actions on response on any of the participants
  - E.g. P is in READY state, asks other “Q” participants

State of Q	Action by P
COMMIT	Make transition to COMMIT
ABORT	Make transition to ABORT
INIT	Make transition to ABORT
READY	Contact another participant

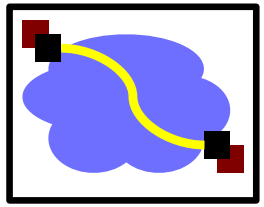
What happens if everyone is in ”READY” state?

Two-Phase Commit (3)

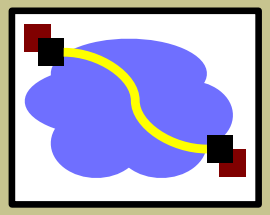


- For recovery, must save state to persistent storage (e.g. log), to restart/recover after failure.
  - Participant (INIT): Safe to local abort, inform Coordinator
  - Participant (READY): Contact others
  - Coordinator (WAIT): Retransmit VOTE_REQ
  - Coordinator (WAIT/Decision): Retransmit VOTE_COMMIT

Summary: Distributed Concurrency



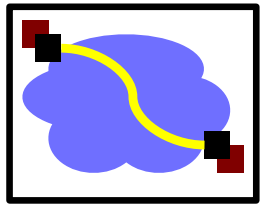
- Distributed consistency management
- ACID Properties desirable
- Single Server case: use locks, and in cases use 2-phase locking (strict 2PL, strong strict 2PL), transactional support for locks
- Multiple server distributed case: use 2-phase commit for distributed transactions. Need a coordinator to manage messages from partipants.



15-440 Distributed Systems

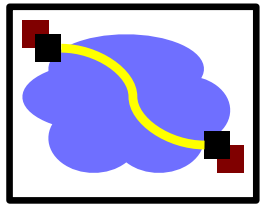
Lecture 10
Fault Tolerance, Logging and recovery
Thursday Oct 8th, 2015

Summary – Fault Tolerance



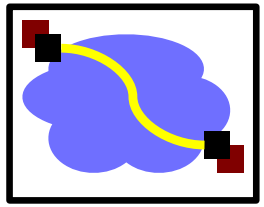
- Real Systems (are often unreliable)
  - Introduced basic concepts for Fault Tolerant Systems including redundancy, process resilience, RPC
- Fault Tolerance – Backward recovery using checkpointing, both Independent and coordinated
- Fault Tolerance –Recovery using Write-Ahead-Logging, balances the overhead of checkpointing and ability to recover to a consistent state

Dependability Concepts



- *Availability* – the system is ready to be used immediately.
- *Reliability* – the system runs continuously without failure.
- *Safety* – if a system fails, nothing catastrophic will happen. (e.g. process control systems)
- *Maintainability* – when a system fails, it can be repaired easily and quickly (sometimes, without its users noticing the failure). (also called Recovery)
  - What's a failure? : System that cannot meet its goals => faults
  - Faults can be: Transient, Intermittent, Permanent

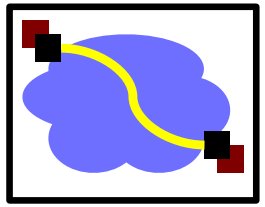
Masking Failures by Redundancy



- **Strategy**: hide the occurrence of failure from other processes using *redundancy*.

1. *Information Redundancy* – add extra bits to allow for error detection/recovery (e.g., Hamming codes and the like).
2. *Time Redundancy* – perform operation and, if needs be, perform it again. Think about how transactions work (BEGIN/END/COMMIT/ABORT).
3. *Physical Redundancy* – add extra (duplicate) hardware and/or software to the system.

Recovery Strategies



- When a failure occurs, we need to bring the system into an error free state (recovery). This is fundamental to Fault Tolerance.

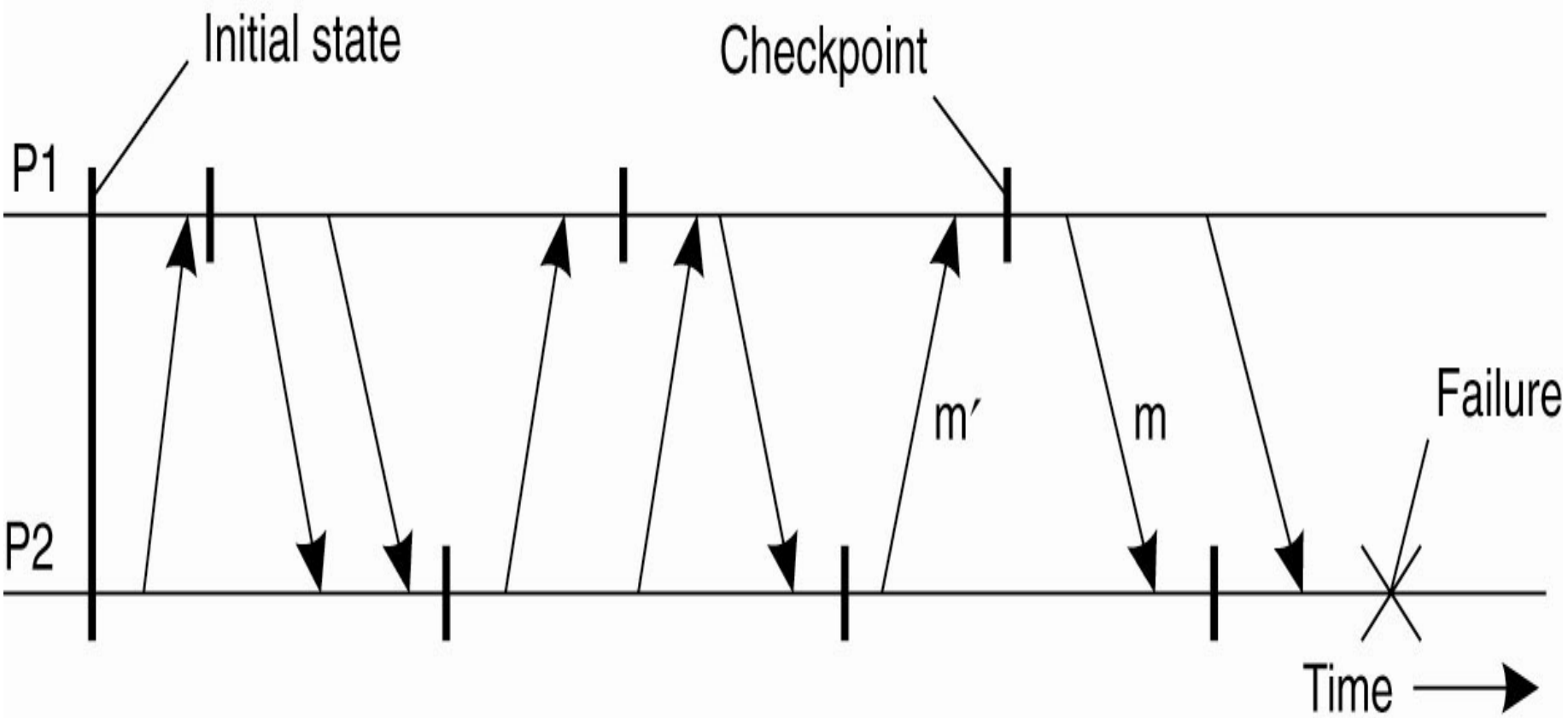
1. **Backward Recovery**: return the system to some previous correct state (using *checkpoints*), then continue executing.

   -- Can be expensive, however still used

2. **Forward Recovery**: bring the system into a correct new state, from which it can then continue to execute.

   -- Need to know potential errors up front!

Independent Checkpointing

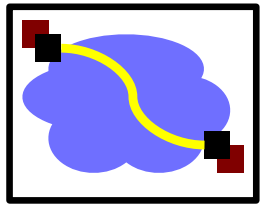


The domino effect – Cascaded rollback

P2 crashes, roll back, but 2 checkpoints inconsistent (P2 shows m received, but P1 does not show m sent)

Solution? Co-ordinated checkpointing

Coordinated Checkpointing

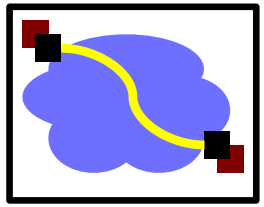


- Key idea: each process takes a checkpoint after a globally coordinated action. (why is this good?)

- Simple Solution: 2-phase blocking protocol
  - Co-ordinator multicast *checkpoint_REQUEST* message
  - Participants receive message, takes a checkpoint, stops sending (application) messages, and sends back *checkpoint_ACK*
  - Once all participants ACK, coordinator sends *checkpoint_DONE* to allow blocked processes to go on

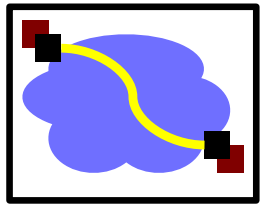
- Optimization: consider only processes that depend on the recovery of the coordinator (those it sent a message since last checkpoint)

Shadow Paging Vs WAL



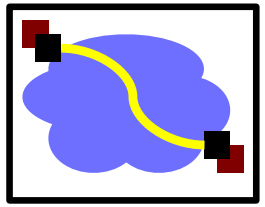
- Shadow Pages
  - Provide Atomicity and Durability, “page” = unit of storage
  - Idea: When writing a page, make a “shadow” copy
    - No references from other pages, edit easily!
  - ABORT: discard shadow page
  - COMMIT: Make shadow page “real”. Update pointers to data on this page from other pages (recursive). Can be done atomically

Shadow Paging vs WAL

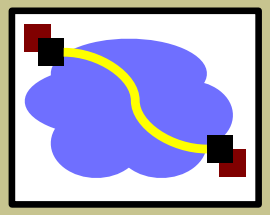


- Write-Ahead-Logging
  - Provide Atomicity and Durability
  - Idea: create a log recording every update to database
  - Updates considered reliable when stored on disk
  - Updated versions are kept in memory (page cache)
  - Logs typically store both REDO and UNDO operations
  - After a crash, recover by replaying log entries to reconstruct correct state
  - 3 Passes: (Analysis Pass, recovery pass, Undo Pass)
  - WAL is more common, fewer disk operations, transactions considered committed once log written.

Recovery using WAL – 3 passes



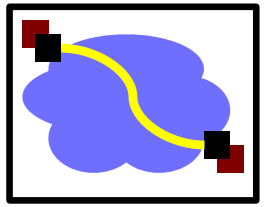
- Analysis Pass
  - Reconstruct TT and DPT (from start or last checkpoint)
  - Get copies of all pages at the start
- Recovery Pass (redo pass)
  - Replay log forward, make updates to all dirty pages
  - Bring everything to a state at the time of the crash
- Undo Pass
  - Replay log file backward, revert any changes made by transactions that had not committed (use PrevLSN)
  - For each write Compensation Log Record (CLR)
  - Once you reach BEGIN TXN, write an END TXN entry



Distributed Replication

Lecture 11, Oct 6th 2016

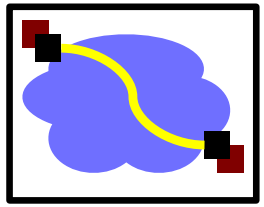
Sequential Consistency (1)





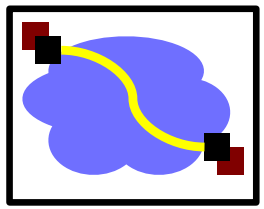
- Behavior of two processes operating on the same data item. The horizontal axis is time.
- P1: Writes “W” value a to variable “x”
- P2: Reads `NIL’ from “x” first and then `a’

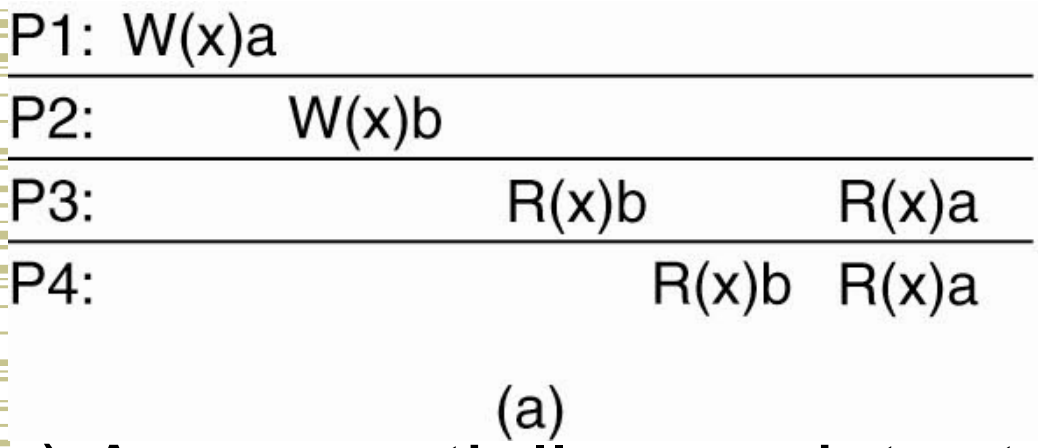
Sequential Consistency (2)



- A data store is sequentially consistent when:
- The result of any execution is the same as if the (read and write) operations by all processes on the data store …
  - Were executed in some sequential order and …
  - the operations of each individual process appear …
    - in this sequence
    - in the order specified by its program.

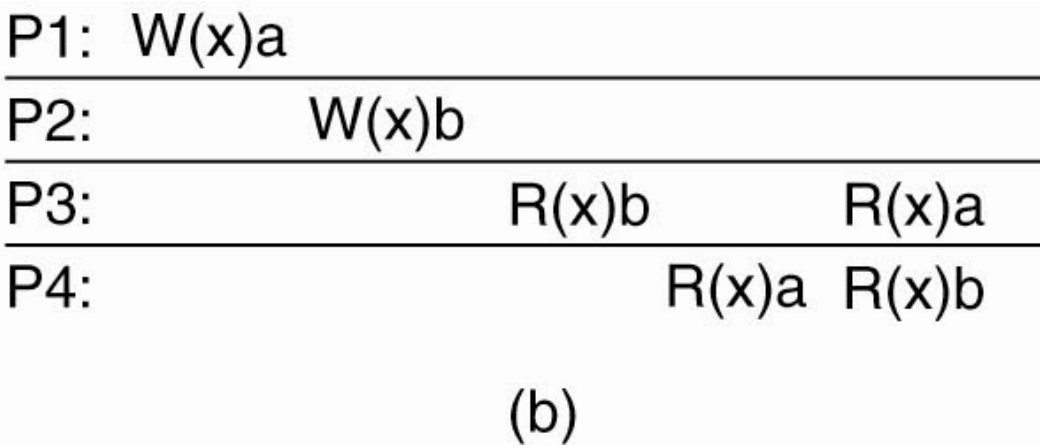
Sequential Consistency (3)





(a)

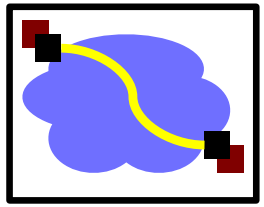
(a) A sequentially consistent data store.



(b)

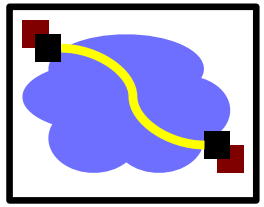
(b) A data store that is not sequentially consistent.

Causal Consistency (1)



- For a data store to be considered causally consistent, it is necessary that the store obeys the following condition:
- Writes that are potentially causally related …
  - must be seen by all processes
  - in the same order.
- Concurrent writes …
  - may be seen in a different order
  - on different machines.

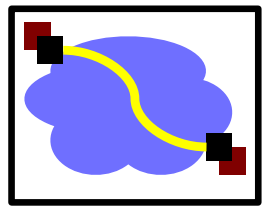
Causal Consistency (2)



P1:	W(x)a			W(x)c	
P2:		R(x)a	W(x)b		
P3:		R(x)a		R(x)c	R(x)b
P4:		R(x)a		R(x)b	R(x)c

- Figure 7-8. This sequence is allowed with a causally-consistent store, but not with a sequentially consistent store.

Replicate: State versus Operations



Possibilities for what is to be propagated:

- Propagate only a notification of an update.

- Sort of an “invalidation” protocol

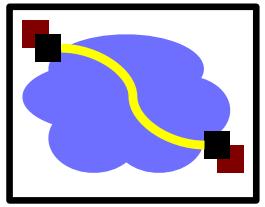
- Transfer data from one copy to another.

- Read-to-Write ratio high, can propagate logs (save bandwidth)

- Propagate the update operation to other copies

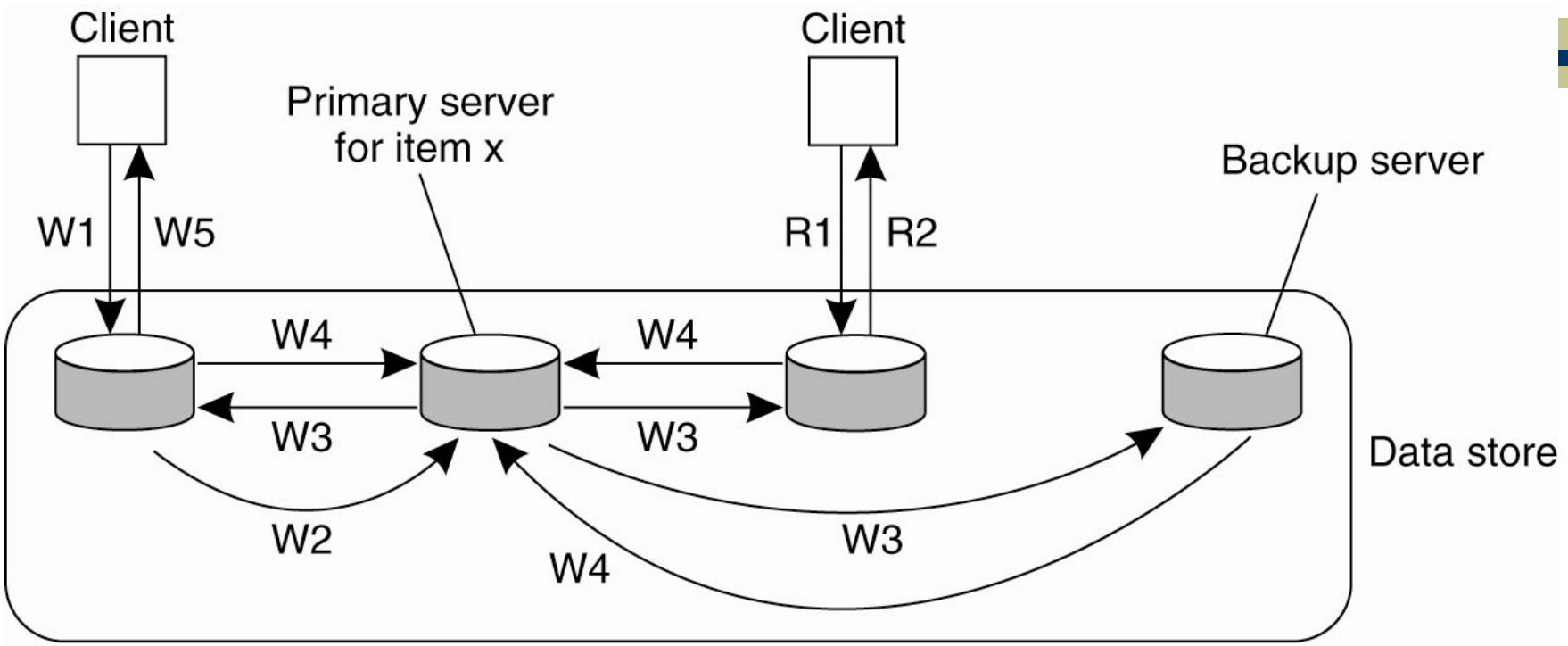
- Don’t transfer data modifications, only operations – “Active replication”

Replication: Primary-Backup



- Clients talk to a primary
- The primary handles requests, atomically and idempotently, just like your lock server would
- Executes them
- Sends the request to the backups
- Backups reply, “OK”
- ACKs to the client

Remote-Write PB Protocol

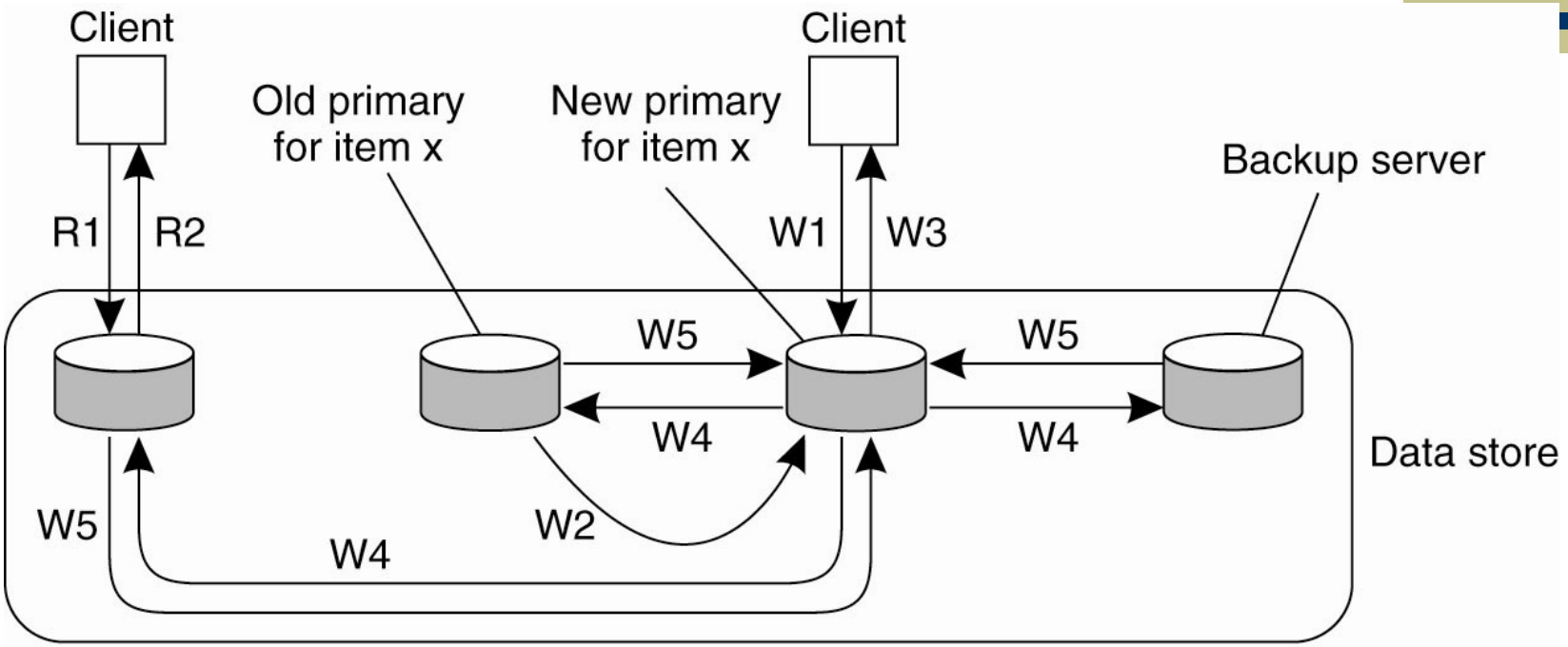


W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

Updates are blocking, although non-blocking possible

Local-Write P-B Protocol



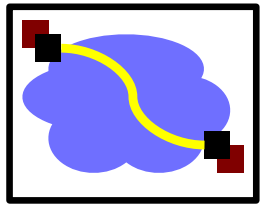
W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

Primary migrates to the process wanting to process update
For performance, use non-blocking op.

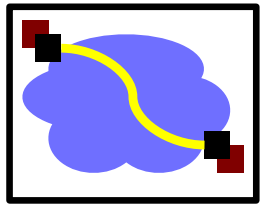
What does this scheme remind you of?

Primary-Backup



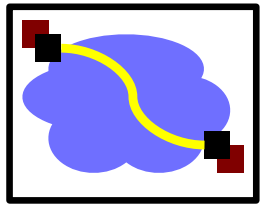
- Note: If you don't care about strong consistency (e.g., the "mail read" flag), you can reply to client *before* reaching agreement with backups (sometimes called "asynchronous replication").
- This looks cool. What's the problem?
  - What do we do if a replica has failed?
  - We wait... how long? Until it's marked dead.
  - Primary-backup has a strong dependency on the failure detector
- This is OK for some services, not OK for others
- Advantage: With N servers, can tolerate loss of N-1 copies

Problems with p-b



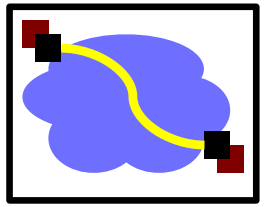
- Not a great solution if you want very tight response time even when something has failed: Must wait for failure detector
- For that, *quorum* based schemes are used
- As name implies, different result:
- To handle *f* failures, must have 2*f* + 1 replicas (so that a majority is still alive)
- Also, for replicated-write => write to all replica's not just one.

Replication: Quorum based consensus



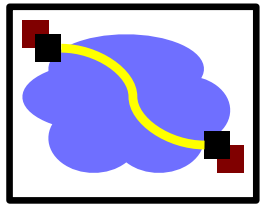
- Quorum consensus
  - Designed to have fast response time even under failures
  - Replicas are “active” - participate in protocol; there is no master, per se.
  - Good: Clients don’t even see the failures. Bad: More complex.

PAXOS: Requirement



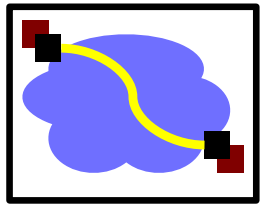
- Correctness (safety):
  - All nodes agree on the same value
  - The agreed value X has been proposed by some node
- Fault-tolerance:
  - If less than N/2 nodes fail, the rest should reach agreement *eventually w.h.p*
  - Liveness is not *guaranteed*
- *Termination (not guaranteed)*

Fischer-Lynch-Paterson [FLP'85] impossibility result



- It is impossible for a set of processors in an asynchronous system to agree on a binary value, even if only a single processor is subject to an unannounced failure.
- Synchrony --> bounded amount of time node can take to process and respond to a request Asynchrony --> timeout is not perfect

Paxos: general approach

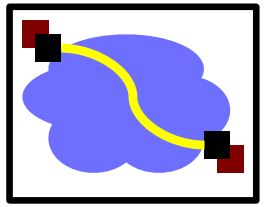


- Elect a replica to be the Leader
- Leader proposes a value and solicits acceptance from others
- If a majority ACK, the leader then broadcasts a commit message.

- This process may be repeated many times, as we'll see.

Paxos slides adapted from Jinyang Li, NYU; some terminology from "Paxos Made Live" (Google)

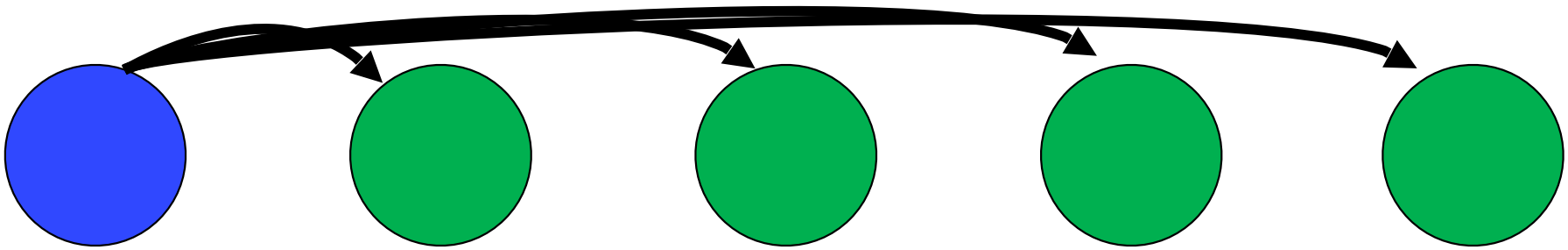
Basic two-phase



- Coordinator tells replicas: “Value V”
- Replicas ACK
- Coordinator broadcasts “Commit!”

- This isn’t enough
  - What if there’s more than 1 coordinator at the same time? (let’s solve this first)
  - What if some of the nodes or the coordinator fails during the communication?

Combined leader election and two-phase



Prepare(N) -- dude, I'm the master

if N >= nH, Promise(N) –
ok, you're the boss. (I haven't seen anyone with a higher N)

if **majority** promised: Accept(V, N) -- please agree on the value V

if N >= nH, ACK(V, N) -- Ok!
if majority ACK: Commit(V)

Multiple coordinators

- The value N is basically a lamport clock.
- Nodes that want to be the leader generate an N higher than any they've seen before
- If you get NACK'd on the propose, back off for a while - someone else is trying to be leader
- Have to check N at later steps, too, e.g.:
- L1: N = 5 --> propose --> promise
- L2: N = 6 --> propose --> promise
- L1: N = 5 --> accept(V1, ...)
- Replicas: NACK! Someone beat you to it.
- L2: N = 6 --> accept(V2, ...)
- Replicas: Ok!

But...

- What happens if there's a failure? Let's say the coordinator crashes before sending the commit message
- Or only one or two of the replicas received it

Paxos solution

- Proposals are ordered by proposal #
- Each acceptor may accept multiple proposals
  - –If a proposal with value `v' is chosen, all higher proposals must have value `v'

Paxos operation: node state

- Each node maintains:
  - n_a, v_a: highest proposal # and its corresponding accepted value
  - n_h: highest proposal # seen
  - my_n: my proposal # in current Paxos

Paxos operation: 3-phase protocol

- Phase 1 (Prepare)
  - –A node decides to be leader (and propose)
  - –Leader chooses $my_n > n_h$
  - –Leader sends <prepare, my_n> to all nodes
  - –Upon receiving <prepare, n>

```
If n < nh
    reply <prepare-reject>
Else
    nh = n
    reply <prepare-ok, na,va>
```

This node will not accept any proposal lower than n

See the relation to lamport clocks?

Paxos operation

- Phase 2 (Accept):
  - –If leader gets prepare-ok from a majority

    V = non-empty value corresponding to the highest n_a received

    If V= null, then leader can pick any V

    Send <accept, my$n$, V> to all nodes
  - –If leader fails to get majority prepare-ok
    - Delay and restart Paxos
  - –Upon receiving <accept, n, V>

    If n < n_h

    reply with <accept-reject>

    else

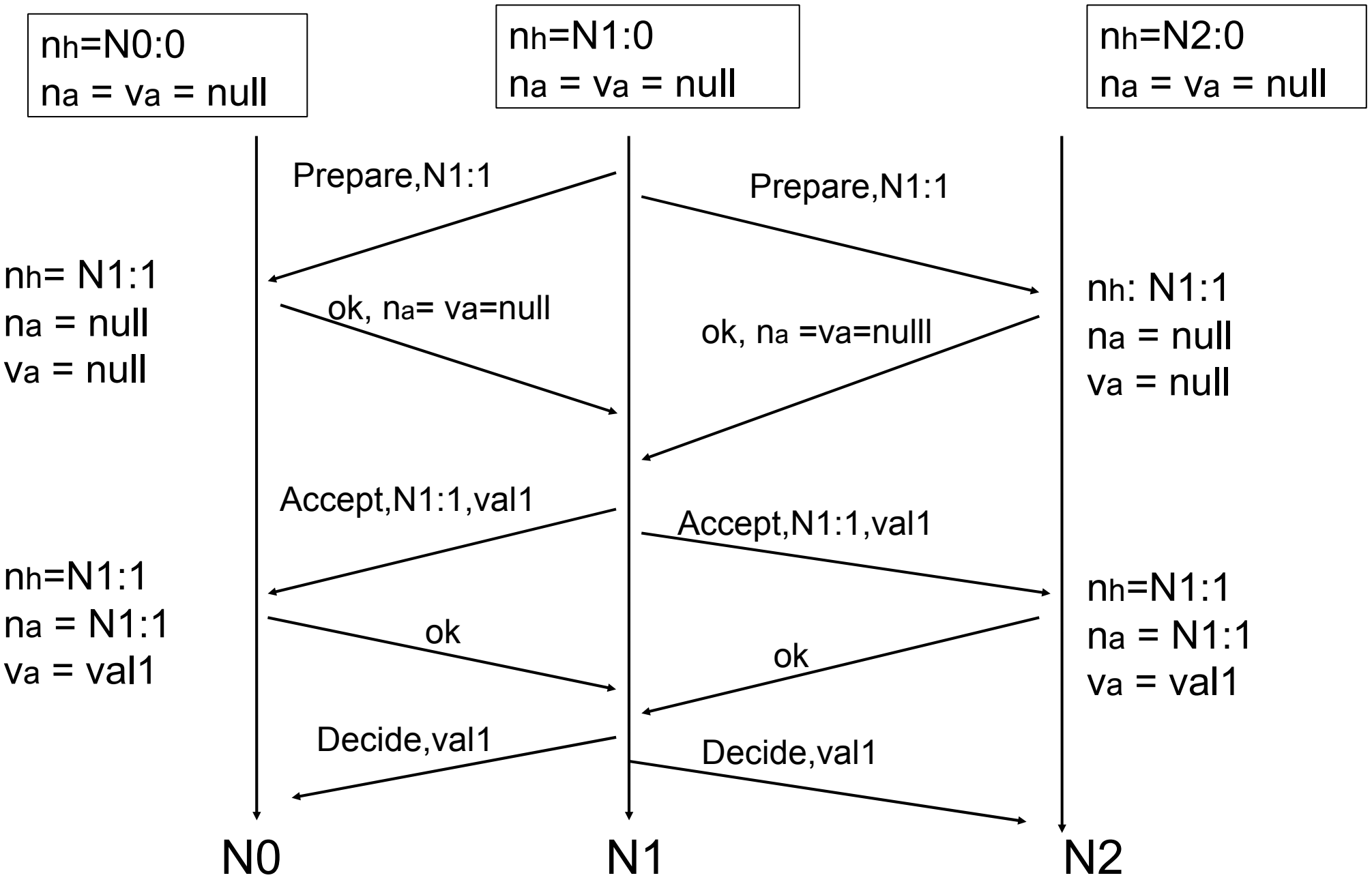
    n_a = n; v_a = V; n_h = n

    reply with <accept-ok>

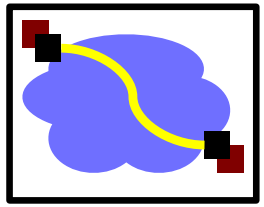
Paxos operation

- Phase 3 (Commit)
  - –If leader gets accept-ok from a majority
    - Send <commit, v_a> to all nodes
  - –If leader fails to get accept-ok from a majority
    - Delay and restart Paxos

Paxos operation: an example

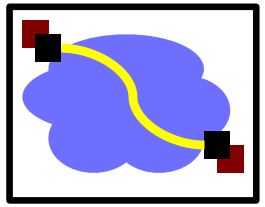


Replication Wrap-Up

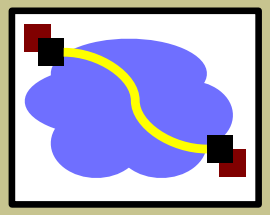


- Primary/Backup quite common, works well, introduces some time lag to recovery when you switch over to a backup. Doesn't handle as large a set of failures. f+1 nodes can handle f failures.
- Paxos is a general, quorum-based mechanism that can handle lots of failures, still respond quickly. 2f+1 nodes.

Beyond PAXOS



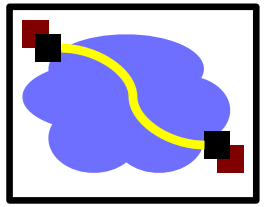
- Many follow ups and variants
- RAFT consensus algorithm
  - https://raft.github.io/
  - Great visualization of how it work
    - http://thesecretlivesofdata.com/raft/



15-440 Distributed Systems

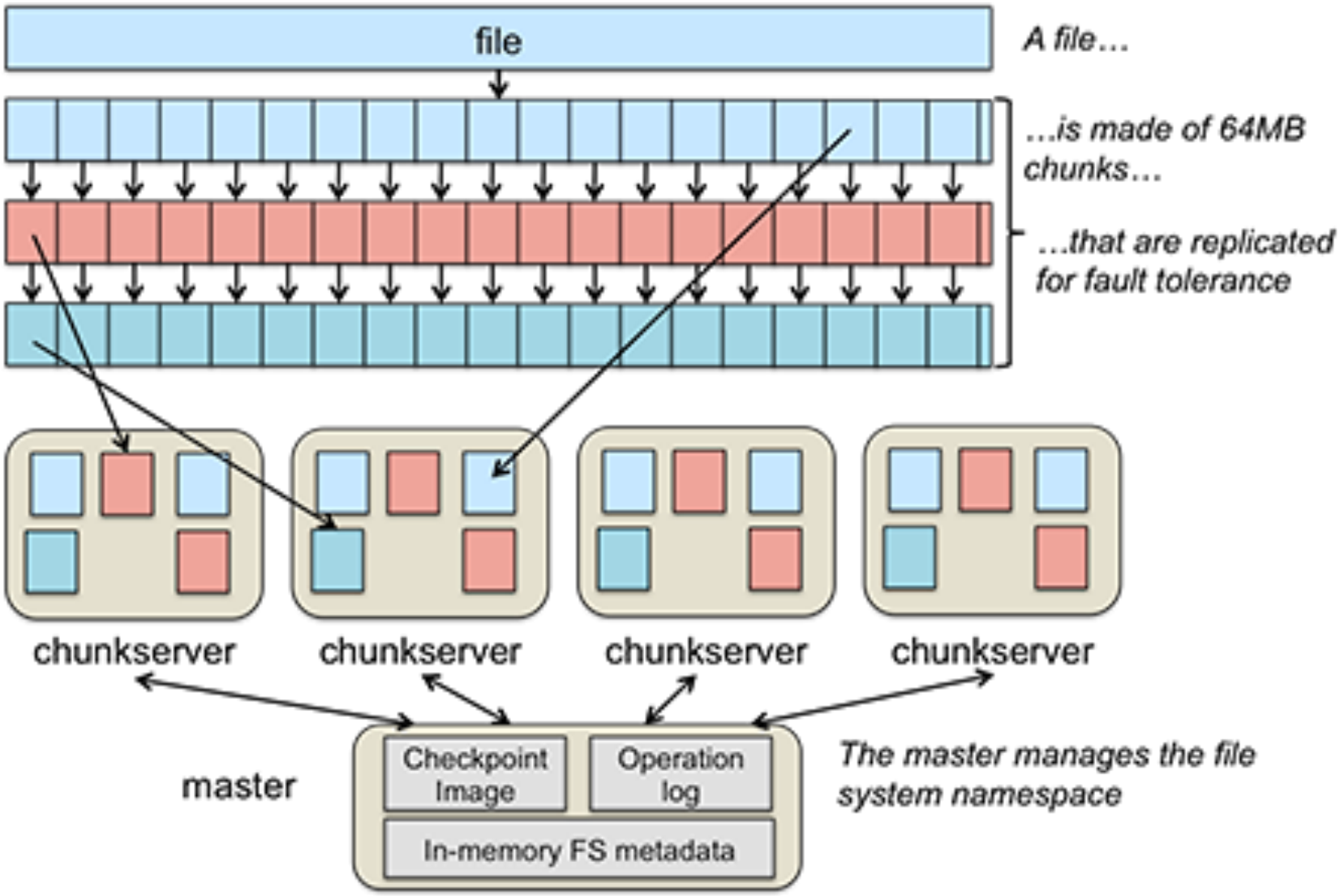
GFS/BigTable/Spanner

Google File System: Design Criteria & Assumptions

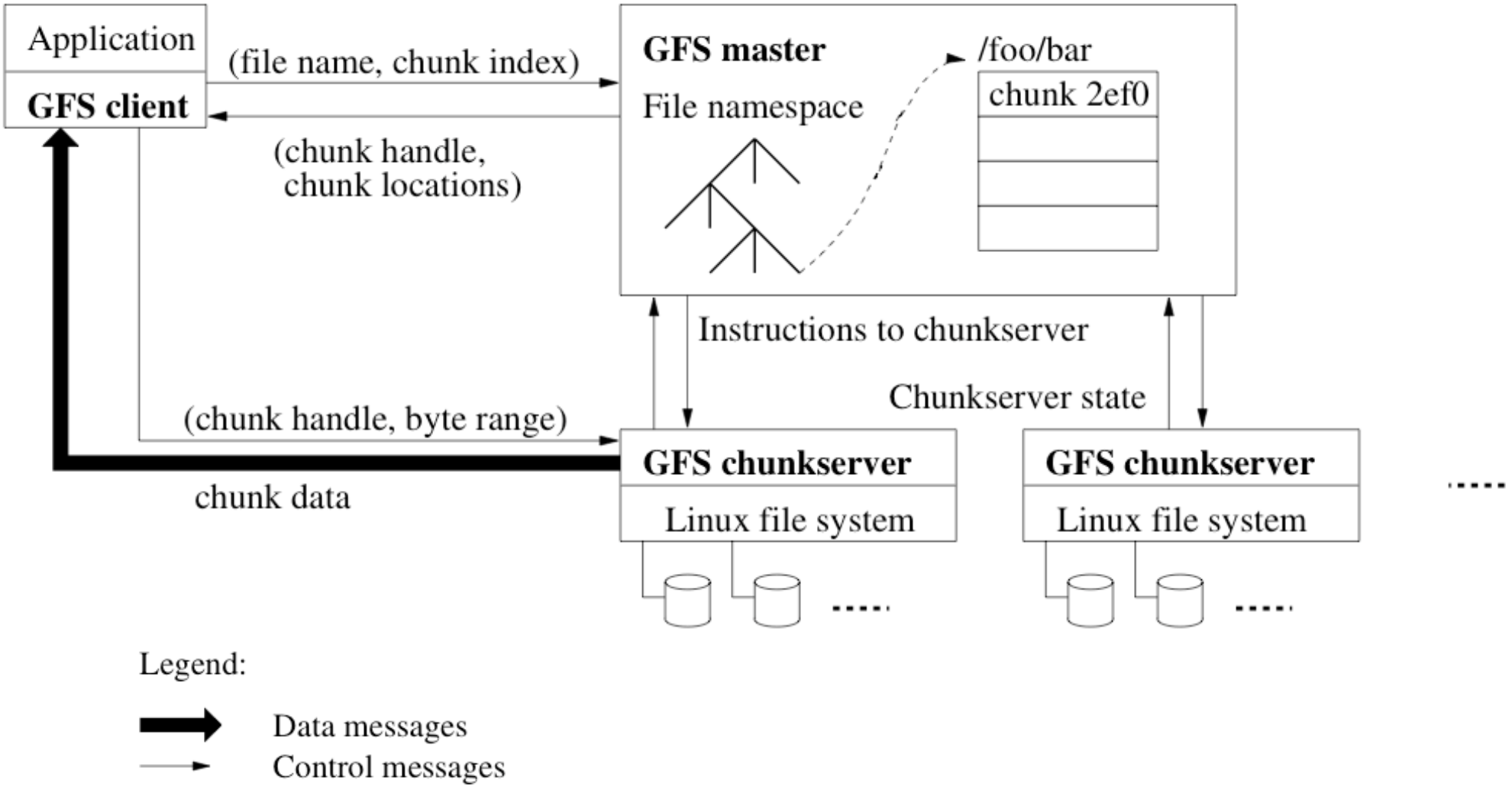


- Detect, tolerate, recover from failures automatically
- Large files, >= 100 MB in size
- Large, streaming reads (>= 1 MB in size)
  - Read once
- Large, sequential writes that append
  - Write once
- Concurrent appends by multiple clients (e.g., producer-consumer queues)
  - Want atomicity for appends without synchronization overhead among clients

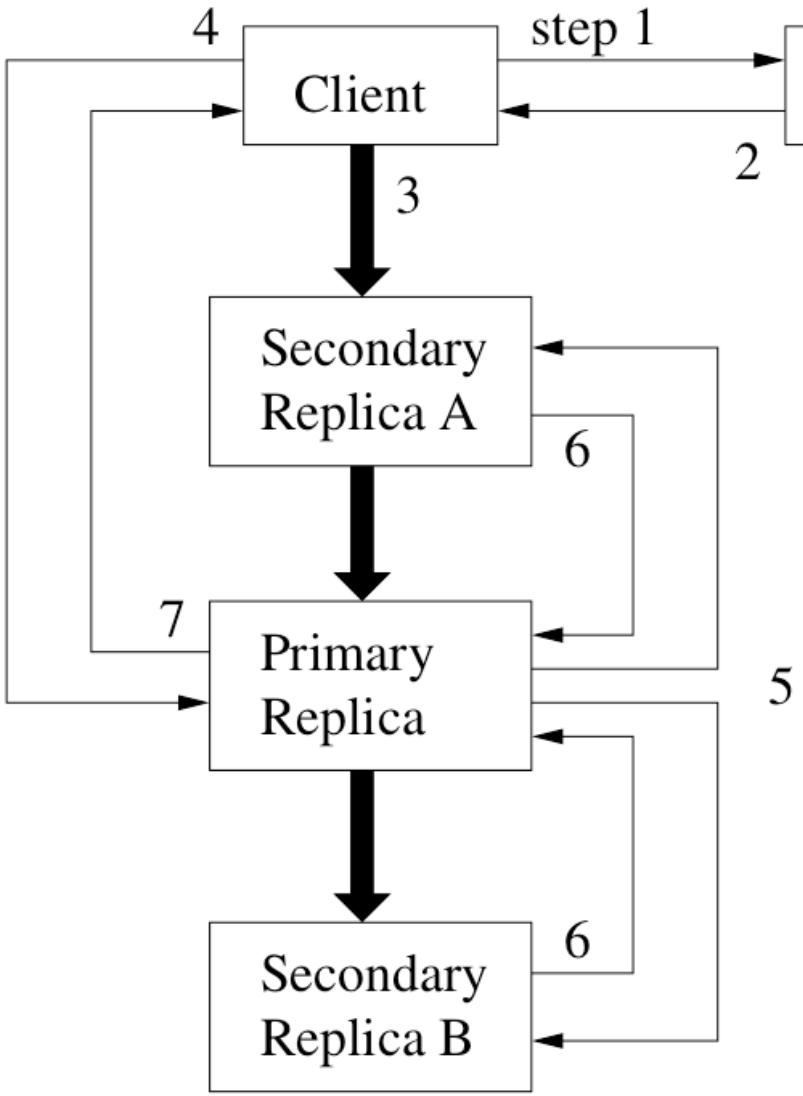
GFS Architecture File Layout



Client Read


Application
GFS client
(file name, chunk index)
(chunk handle,
chunk locations)
GFS master
File namespace
/foo/bar
chunk 2ef0
Instructions to chunkserver
Chunkserver state
(chunk handle, byte range)
chunk data
GFS chunkserver
Linux file system
GFS chunkserver
Linux file system
Legend:
Data messages
Control messages

Client Write (2)



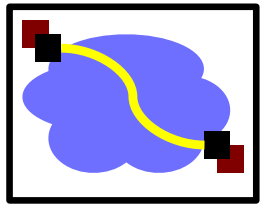
Send to closest replica first

Legend:

Control

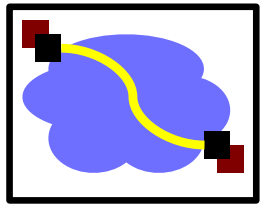
Data

Client Record Append



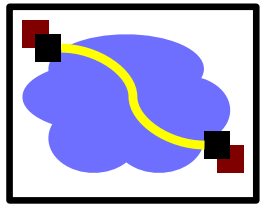
- **Append**: allows a client to append data to a file as an atomic operation without having to lock a file. Multiple processes can append to the same file concurrently without fear of overwriting one another's data
- Same control flow as for writes, except…
- Client pushes data to replicas of last chunk of file
- Client sends request to primary
- Common case: request fits in current last chunk:
  - Primary appends data to own replica
  - Primary tells secondaries to do same at same byte offset in theirs
  - Primary replies with success to client

GFS: Consistency Model (2)

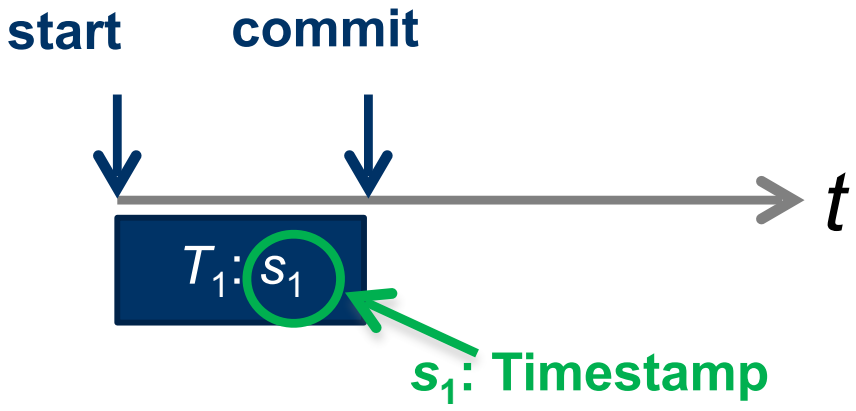


- Changes to data are ordered as chosen by a primary
  - But multiple writes from the same client may be interleaved or overwritten by concurrent operations from other clients
- Record append completes at least once, at offset of GFS's choosing
  - Applications must cope with possible duplicates
- Failures can cause inconsistency
  - Behavior is worse for writes than appends

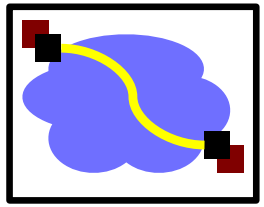
Spanner Concurrency Control



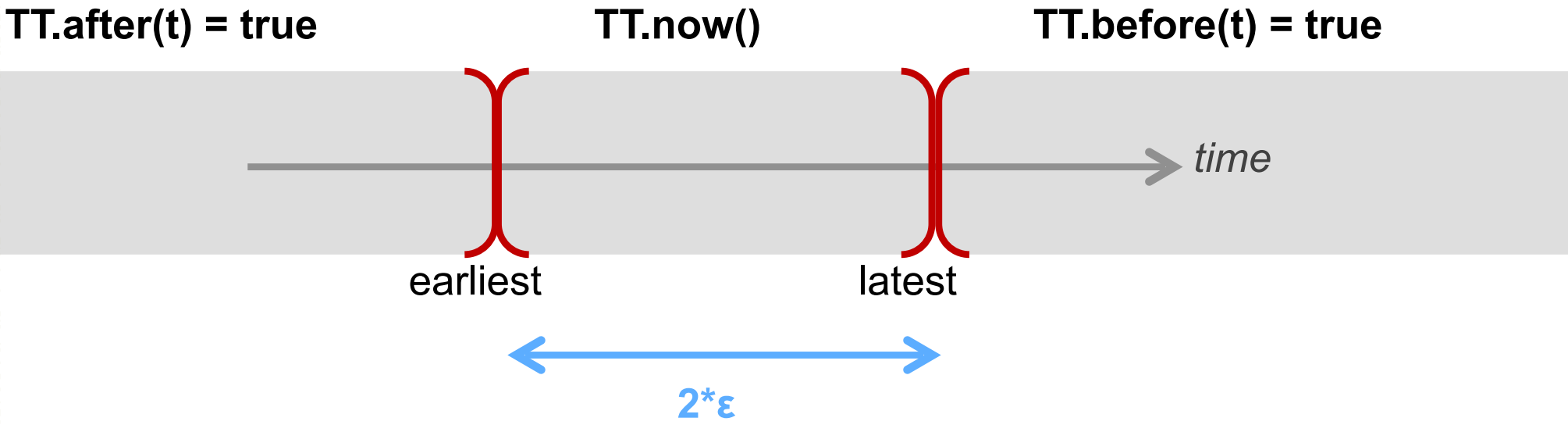
- Key aspect of differentiating Spanner – using globally meaningful timestamps for distributed transactions in achieving external consistency



Spanner TrueTime API

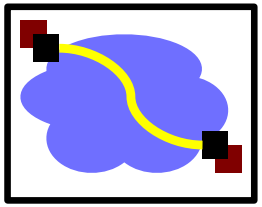


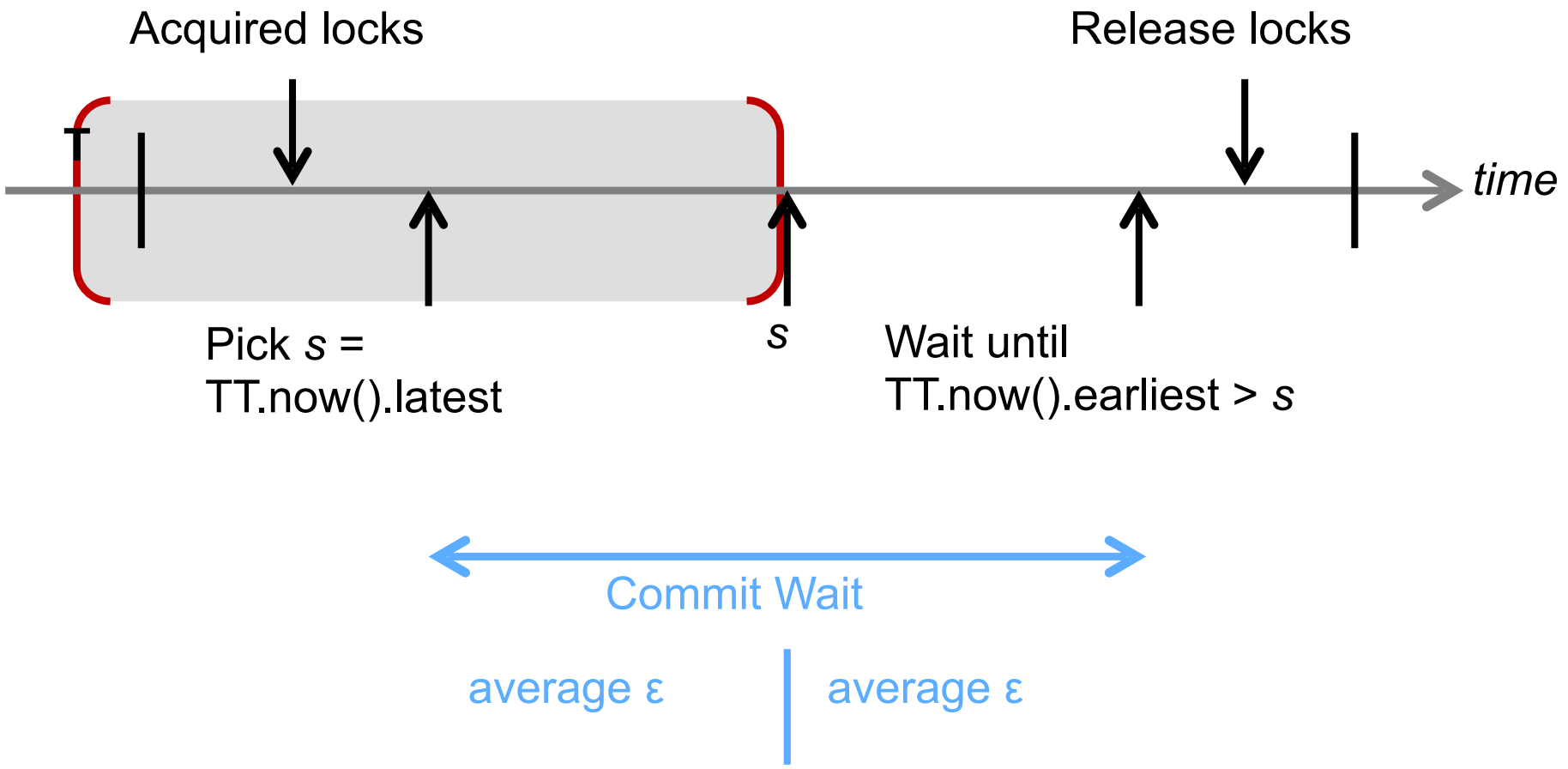
TT.now() – TTInterval: [earliest, latest]
(Why is time an interval and not a specific instant?)
TT.after(t) – true if **t** has definitely passed
TT.before(t) – true if **t** has definitely not arrived



"Global wall-clock time" with bounded uncertainty

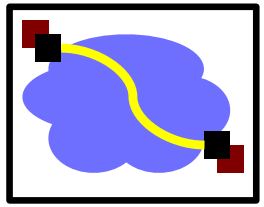
Spanner Commit Wait





Why do you need to wait for TT.now().earliest > s before releasing locks?

Spanner External Consistency



- If a transaction T_1 commits before another transaction T_2 starts, then T_1's commit timestamp is smaller than T_2
- Similar to how we reason with wall-clock time

